

Tailorable codes for lattice-based KEMs with applications to compact ML-KEM instantiations

Thales B. Paiva¹, Marcos A. Simplicio Jr^{1,2}, Syed Mahbub Hafiz¹,
Bahattin Yildiz¹, Eduardo L. Cominetti¹ and Henrique S. Ogawa¹

¹ Future Security Team, LG Electronics, Santa Clara, USA

² Universidade de São Paulo, São Paulo, Brazil

{thalespaiva, msimplicio, ecominetti}@larc.usp.br

{syedmahbub.hafiz, bahattin.yildiz, henrique1.ogawa}@lge.com

Abstract. Compared to elliptic curve cryptography, a primary drawback of lattice-based schemes is the larger size of their public keys and ciphertexts. A common procedure for compressing these objects consists essentially of dropping some of their least significant bits. Albeit effective for compression, there is a limit to the number of bits to be dropped before we get a noticeable decryption failure rate (DFR), which is a security concern. To address this issue, this paper presents a family of error-correction codes that, by allowing an increased number of dropped bits while preserving a negligible DFR, can be used for both ciphertext and public-key compression in modern lattice-based schemes. To showcase the impact and practicality of our proposal, we use the highly optimized ML-KEM, a post-quantum lattice-based scheme recently standardized by NIST. We provide detailed procedures for tailoring our codes to ML-KEM’s specific noise distributions, and show how to analyze the DFR without independence assumptions on the noise coefficients. Among our results, we achieve between 4% and 8% ciphertext compression for ML-KEM. Alternatively, we obtain 8% shorter public keys compared to the current standard. We also present isochronous implementations of the decoding procedure, achieving negligible performance impact in the full ML-KEM decapsulation even when considering optimized implementations for AVX2, Cortex-M4, and Cortex-A53.

Keywords: PQC · ML-KEM · Error correction codes · Ciphertext compression

1 Introduction

In 2022, NIST chose Kyber [7], a lattice-based key-encapsulation mechanism (KEM), for standardization as ML-KEM [32]. Like most modern quantum-resistant lattice-based schemes, ML-KEM is very efficient, but the sizes of its public keys and ciphertexts are orders of magnitude larger than those of schemes based on elliptic curves. Although it is possible to compress public keys and ciphertexts by dropping a few of their least significant bits, there is a limit to how much one can compress them before decryption failures start to occur. Such failures are a well-known security concern [16, 20, 24] and can be exploited by attackers to mount key-recovery attacks. Therefore, designers of lattice-based schemes usually employ error-correction strategies aiming to achieve negligible DFR even when compression techniques are used.

Following Regev’s [36] work, ML-KEM and other efficient lattice-based schemes use the same encoding scheme during encryption: each bit b of the message is encoded into $\mathbb{Z}_q$ as $b\lceil q/2 \rceil$. For higher performance, though, some schemes take extra steps to achieve better error correction. For example, some lattice-based candidates in the first round of

NIST’s post-quantum standardization process [1] apply distinct error-correction codes to the message before encryption. LAC [30] uses well-known BCH codes, Round5 [8] uses a custom code named XEf [37], and NewHope [2] uses repetition codes. Interestingly, a previous version of NewHope [3] used more complex, 4-dimensional lattice codes, but those were replaced by simpler repetition codes, which are easier to understand and analyze.

In an effort to improve Kyber’s performance, recent studies have explored ways to adapt Kyber for the use of higher-dimensional lattice codes [29, 38, 39]. Unfortunately, state-of-art techniques still come with significant limitations. For example, Liu and Sakzad’s [29] approach assumes that the coefficients of the noise polynomial accumulated during decryption are independent, which does not hold in practice and may result in an overestimation of the scheme’s security [16]. Meanwhile, although the work by Saliba et al. [38, 39] does not require independence assumptions, the resulting Kyber variant has a larger ciphertext size. Moreover, all of these approaches [29, 38, 39] require changing at least one of Kyber’s core parameters: the polynomial degree n and the modulus q . Consequently, the resulting constructions cannot take advantage of Kyber’s NTT-based efficient polynomial multiplications, a major feature behind the scheme’s high performance.

Contributions. We present a new family of higher-dimensional error-correction codes, called Minal codes, that are applicable to most modern lattice-based KEMs, including ML-KEM [32], Saber [15], and NewHope [2]. Our codes are designed to be efficiently decodable in small dimensions and have two main benefits. First, unlike other complex encoding schemes, Minal codes do not impose limitations on the underlying scheme parameters (e.g., power-of-two moduli are not required). Second, our codes can be tailored to the specific distribution of the error to be corrected for each different target scheme.

To demonstrate the practicality of our proposal in a concrete and challenging scenario, we target ML-KEM, which is arguably the most important and highly optimized lattice-based KEM available today. We show that Minal codes allow 4% to 8% shorter ciphertexts for ML-KEM, without adding independence assumptions or requiring changes to the scheme’s core parameters (unlike [29, 38, 39]), while maintaining ML-KEM’s DFR close to the values targeted by the current standard. Actually, only our proposal for Level 3 requires one extra assumption, namely the hardness of the learning with rounding (LWR), which is used by ML-KEM-512 (but not for the higher levels). Alternatively, our codes can be used to obtain 8% compression of public keys in all security levels. We also discuss how Minal codes could enable previously unexplored parameter sets by lowering their DFRs to negligible values. In addition, unlike previous work, we show such gains with isochronous implementations for encoding and decoding, reinforcing the practical benefits of our codes.

The performance impact of our codes on ML-KEM’s decapsulation is evaluated by considering optimized implementations for AVX2 or running on Cortex M4 and A53 processors. In most of the platforms considered, the impact on decapsulation remains below 1%, much lower than the benefits in terms of ciphertext or public key compression. Our code and data are publicly available at https://github.com/thalespaiva/minal_mlkem.

2 Background and setup

Notation and background. For any prime q , we write $\mathbb{Z}_q$ to denote the field of integers modulo q . When n is a fixed positive integer, we let R_q denote the polynomial ring $\mathbb{Z}_q[x]/(x^n + 1)$. Then, R_q^k is the module of rank k whose scalars are polynomials in R_q . Polynomials $a \in R_q$ are denoted using lowercase letters. Vectors $\mathbf{a} \in R_q^k$ and matrices $\mathbf{A} \in R_q^{k \times k}$ are denoted in bold using lowercase and uppercase, respectively, where $k \geq 1$. When $\mathbf{u}, \mathbf{v} \in R_q^k$, we let $\langle \mathbf{u}, \mathbf{v} \rangle \in R_q$ denote their dot product.

Given a polynomial $a \in R_q$, the function `poly_to_vec` returns its n coefficients as a vector in $\mathbb{Z}_q^n$. In other words, given $a = a_0 + a_1x + \dots + a_{n-1}x^{n-1}$, we have

$\mathbf{a} = \text{poly_to_vec}(a) = [a_0, a_1, \dots, a_{n-1}]$. With a slight abuse of notation, we denote the i -th coefficient of a polynomial $a \in R_q$, associated with the power x^i , by either a_i (when discussing the polynomial form of a) or by $a[i]$ (when discussing its vector equivalent). Tuples of coefficients from a polynomial a are denoted by $a[i_1, \dots, i_n] = (a[i_1], \dots, a[i_n])$. We denote by $\mathcal{B}_\eta$ the centered binomial distribution (CBD) with range $[-\eta, \eta]$.

The circulant matrix generated by a vector $\mathbf{a} \in \mathbb{Z}^n$ is the $n \times n$ matrix whose first row is $\mathbf{a}$, and each subsequent row is a right circular shift of the row above it. Let negashift_i be the function that returns the i -th column of the negacyclic matrix generated by the coefficients of a given polynomial. For example, if $a = a_0 + a_1x + \dots + a_{n-1}x^{n-1}$, then $\text{negashift}_i(a) = [a_i, \dots, a_0, -a_{n-1}, \dots, -a_{i+1}]$. With this notation, we can represent the product of polynomials a and b in the negacyclic ring R_q using its vector form, whose i -th coefficient is given by $\text{poly_to_vec}(ab)[i] = \langle \text{poly_to_vec}(a), \text{negashift}_i(b) \rangle$.

The centered modulo operation, denoted as $a' = a \bmod^\pm q$, returns the unique integer $a' \equiv a \bmod q$ such that $-\lfloor (q-1)/2 \rfloor \leq a' \leq \lfloor (q-1)/2 \rfloor$. The distance modulo q between points $\mathbf{v}_1$ and $\mathbf{v}_2$ is defined as $\text{dist}_q(\mathbf{v}_1, \mathbf{v}_2) = \|(\mathbf{v}_1 - \mathbf{v}_2) \bmod^\pm q\|$. We write $y = \text{Compress}(x, d)$ to denote the lossy compression of x to d bits defined as $\text{Compress}(x, d) = \lfloor (2^d/q)x \rfloor \bmod 2^d$, where $\lfloor \cdot \rfloor$ is the rounding function that rounds up on ties. The decompression is defined as $x' = \text{Decompress}(y, d) = \lfloor (q/2^d)y \rfloor$. The error $|x' - x|$ caused by (de)compression is then almost uniform over the set $\{-\lfloor q/2^{d+1} \rfloor, \dots, \lfloor q/2^{d+1} \rfloor\}$, with possibly some slight skewness when q is not a power of 2.

Experimental setup. To ensure a broad evaluation, our experimental testbed covers three types of platforms: Intel AVX2, ARM Cortex-A53, and ARM Cortex-M4.

To obtain the AVX2 cycle counts, we considered the latest AVX2 implementation provided by the Kyber team [7] running on an Intel Core i7-8700 (Coffee Lake) CPU with a base frequency of 3.20GHz. The code was compiled using gcc version 14.2.1 with flags `-O3, -march=native, -mtune=native`, and `-fomit-frame-pointer`. We report the median cycle count of 10,000 executions.

For Cortex-A53, we used the AArch64-optimized Kyber implementation from PQ-Clean [27, commit 0c5bb14], running on a Raspberry Pi Zero 2W [34]. The compilation was done using `aarch64-none-linux-gnu-gcc v.13.3.1` with flags `-O3, -mcpu=cortex-a53`, and `-mtune=cortex-a53`. For cycle count benchmarking, we used the Performance Monitors Cycle Counter (PMCCNTR_ELO) to measure the average of 10,000 runs.

For Cortex-M4, we used the `m4fspeed` version of `pqm4` [28, commit cda61fb], and measured the performance on an STM Nucleo-F439ZI board [40]. The compilation employed `arm-none-eabi-gcc v.13.2.1` with the following flags: `-O3, -mcpu=cortex-m4, -mfpv4-sp-d16, -mfloat-abi=hard`, and `-mthumb`. For cycle counts, we used the Data Watchpoint and Trace (DWT) registers to get the average of 100 runs.

3 ML-KEM

This section reviews ML-KEM's parameters and algorithms, ending with a discussion on alternative encoding mechanisms that are related to our construction.

3.1 Parameters and algorithms

ML-KEM is a lattice-based key encapsulation mechanism (KEM) whose security relies on the intractability of the module learning with errors (MLWE) problem. Essentially, it enables two parties to establish a shared 256-bit secret. In what follows, we present a slightly simplified version of ML-KEM that, although lacking some details, is sufficient for our discussion. In particular, we describe only the underlying algorithms that make

Table 1. ML-KEM parameters for each security level [32].

NIST security	Parameter set	k	η_1	η_2	d_u	d_v	Ciphertext size (bytes)	DFR
Level 1	ML-KEM-512	2	3	2	10	4	768	$2^{-139.1}$
Level 3	ML-KEM-768	3	2	2	10	4	1088	$2^{-165.2}$
Level 5	ML-KEM-1024	4	2	2	11	5	1568	$2^{-175.2}$

the core of ML-KEM secure against chosen-plaintext attacks (CPA), ignoring the implicit-rejection Fujisaki-Okamoto (FO) transformation that protects the scheme against adaptive chosen-ciphertext attacks (CCA) [23, 26]. Furthermore, we omit the optimizations based on the number theoretic transform (NTT) that are part of the ML-KEM specification.

Setup. ML-KEM supports NIST security levels 1, 3, and 5 [32]. For all security levels, it fixes parameters $q = 3329$ and $n = 256$, defining the polynomial ring $R_q = \mathbb{Z}_q[x]/(x^n + 1)$ over which most operations are performed. This benefits crypto-agility, as any optimization or hardware acceleration for operations in R_q can be reused for all security levels. Given a desired security level, the setup takes public parameters k, η_1, η_2, d_u , and d_v from Table 1: k defines the sizes of the modules used in the scheme; η_1 and η_2 define the centered binomial distributions $\mathcal{B}_{\eta_1}$ and $\mathcal{B}_{\eta_2}$, used to generate coefficients with small norm in $\mathbb{Z}_q$; and integers d_u and d_v are the number of bits into which coefficients from the two parts of the ciphertext are compressed. Table 1 also shows the upper bounds on the decryption failure rate (DFR) for each parameter set, as computed using the Kyber security scripts [19].

Key generation. Let $\mathbf{A}$ be a $k \times k$ matrix of polynomials sampled uniformly at random from R_q . Sample two vectors $\mathbf{s}$ and $\mathbf{e}$ from $\mathcal{B}_{\eta_1}(R_q^k)$, i.e., the coefficients of their polynomials are sampled according to the centered binomial distribution $\mathcal{B}_{\eta_1}$. Compute vector $\mathbf{t} = \mathbf{A}\mathbf{s} + \mathbf{e} \in R_q^k$. The resulting public key is $(\mathbf{A}, \mathbf{t})$, while the private key is the vector $\mathbf{s} \in R_q^k$.

Encryption. Let $\mathbf{m}$ be an n -bit message to be encrypted using public-key $(\mathbf{A}, \mathbf{t})$. Sample vectors $\mathbf{r}$ and $\mathbf{e}_1$ from $\mathcal{B}_{\eta_1}(R_q^k)$ and $\mathcal{B}_{\eta_2}(R_q^k)$, respectively. Similarly, sample a polynomial e_2 from $\mathcal{B}_{\eta_2}(R_q)$. Let $\mathbf{u} = \mathbf{A}^T \mathbf{r} + \mathbf{e}_1$. Compute polynomial $z = \langle \mathbf{t}, \mathbf{r} \rangle + e_2$. Let $v = \text{Encode}(\mathbf{m}) + z$, where the encoding function, when applied to each bit b of $\mathbf{m}$, returns $\text{Encode}(b) = b \lceil q/2 \rceil$. Compress the coefficients of vector $\mathbf{u}$ and polynomial v to d_u and d_v bits, respectively, obtaining $\mathbf{c}_u = \text{Compress}(\mathbf{u}, d_u)$ and $c_v = \text{Compress}(v, d_v)$. Finally, return the ciphertext $(\mathbf{c}_u, c_v)$.

Decryption. To decrypt ciphertext $(\mathbf{c}_u, c_v)$ using secret key $\mathbf{s}$, first decompress the ciphertext components to obtain $\mathbf{u}' = \text{Decompress}(\mathbf{c}_u, d_u)$ and $v' = \text{Decompress}(c_v, d_v)$. Compute $m' = v' - \langle \mathbf{u}', \mathbf{s} \rangle$. Let $\Delta \mathbf{u} = \text{Decompress}(\mathbf{c}_u, d_u) - \mathbf{u}$ and $\Delta v = \text{Decompress}(c_v, d_v) - v$. We can then write $m' = \text{Encode}(\mathbf{m}) + \Delta m$, where the accumulated noise polynomial Δm is given by $\Delta m = \langle \mathbf{e}, \mathbf{r} \rangle - \langle \mathbf{s}, \mathbf{e}_1 + \Delta \mathbf{u} \rangle + e_2 + \Delta v$.

ML-KEM's parameters are carefully chosen to ensure that polynomial Δm has only relatively small coefficients. The message can then be recovered by computing $\hat{m} = \text{Decode}(m')$, where the decoding function, applied to each coefficient of the noisy polynomial m' , returns 0 or 1, depending on whether $m'[i]$ is closer to 0 or to $q/2$, considering distances modulo q . More formally, $\text{Decode}(m'[i]) = 0$ if $|m'[i] \bmod^\pm q| < \lceil q/4 \rceil$, otherwise $\text{Decode}(m'[i]) = 1$.

3.2 Security and decryption failure rate

ML-KEM's design and security analysis revolve around finding parameters that ensure the MLWE problems protecting the secret key and the ciphertext are hard to solve while

Table 2. The DFR and ciphertext sizes obtained by Saliba et al. [38,39].

Security	q	Ciphertext size (bytes)	DFR	Relative ciphertext size	Relative DFR	Advantages (ciphertext size and DFR)
Level 1	2^{11}	832	2^{-133}	108.3%	2^6	None
Level 3	2^{11}	1184	2^{-174}	108.8%	2^{-10}	Better DFR
Level 5	2^{11}	1600	2^{-137}	102.0%	2^{37}	None

maintaining a negligible DFR. To facilitate the scheme’s security analysis, the Kyber team provided public scripts [19] that compute the complexity of known attacks, which is obtained through the core-SVP hardness metric [3], and the resulting DFR.

The parameters having the most impact on the MLWE security are the modulus q , the degree n , and the module dimension k , together with the parameters η_1 and η_2 that control the noise added to the LWE samples. Albeit not as much, the ciphertext compression parameters d_u and d_v can also affect security. In particular, under the learning with rounding (LWR) hardness assumption, increasing the deterministic compression noise improves security. This is explored in the choice of ML-KEM-512 parameters to reduce ciphertext sizes, making it the only parameter set whose security is based both on the hardness of LWE and on an LWR-like assumption.

A necessary condition for a KEM to provide CCA security is to resist attacks exploiting decryption failures [20, 24]. For a given parameter set, the DFR is algorithmically computed as follows. Since each coefficient $\Delta m[i]$ follows the same distribution, we can start by computing the distribution of $\Delta m[0]$. This is done by considering the sums of the distributions corresponding to the right-hand side of the following equation $\Delta m[0] = \langle \mathbf{e}, \mathbf{r} \rangle[0] - \langle \mathbf{s}, \mathbf{e}_1 + \Delta \mathbf{u} \rangle[0] + e_2[0] + \Delta v[0]$, which are easy to compute. Then, an upper bound on the DFR is computed using the union bound as $\Pr(\text{Decryption fails}) \leq n \Pr(|\Delta m[0] \bmod^\pm q| \geq q/4)$. Table 1 shows the DFR values achieved by ML-KEM as computed using the scripts provided by the Kyber team [19].

3.3 Previous work on alternative encoding methods for Kyber

Like our work, some recent studies present strategies to use higher-dimensional codes in Kyber variants. One example is [29], which relies on lattice codes with dimensions 16 and 24. The authors claim to improve both the DFR and the ciphertext size, but they require n to be changed, preventing the NTT-based multiplication. Moreover, their work, like most proposals for error correction in lattice-based schemes, requires independence assumptions on the coefficients of the noise polynomial Δm , which do not hold in practice and are known to cause potentially dangerous underestimation of the DFR when error-correction codes are used [16]. In particular, these assumptions would break Kyber’s DFR arguments from Section 3.2, so it would be hard (if at all feasible) to adapt [29] to Kyber’s design.

More closely related to our work is the approach taken by Saliba et al. [39], which is explained in depth in Saliba’s PhD thesis [38]. They propose a variant of Kyber based on reconciliation, which, in lattice-based schemes, refers to a procedure in which the sender and receiver produce the same shared string from different noisy versions of it. This contrasts with the encoding-decoding paradigm, where the intended shared message is predefined. Their construction uses 8-dimensional lattice codes and does not require independence assumptions, so it can be seen as an extension of the original NewHope’s DFR analysis [3, 33] to Kyber. While Saliba et al. [38, 39] delivers between 10 to 15 extra bits of LWE security for Kyber’s three security levels, their approach has a few practical shortcomings, listed in Table 2. One of the main issues is that the values of the modulus q are powers of two, which means they cannot use the NTT for polynomial multiplication. Furthermore, their scheme increases the ciphertext size in all security levels, while the DFR is increased in levels 1 and 5. For example, compared with Kyber, there is a noticeable

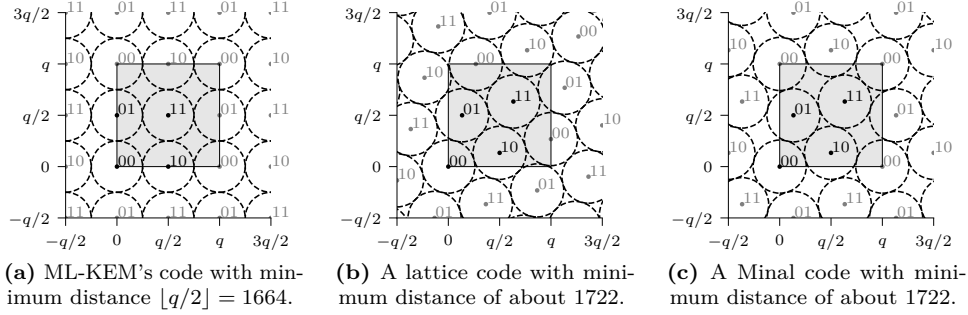


Figure 1. Comparison between the original ML-KEM code seen as a 2D code, a denser lattice code, and a Minal code. The shaded areas represent the $\mathbb{Z}_q^2$ square.

increase in the DFR for level 5, by a factor of 2^{37} .

In summary, since the approaches found in the literature [29, 38, 39] on alternative Kyber encoding mechanisms require changing n or q , they do not benefit from a core feature in Kyber: the fast NTT-based multiplication. Moreover, their proposal's performance is not reported, and we could not find any public implementation for an independent evaluation.

4 Minal codes: Tailorable codes for lattice-based schemes

In this section, we introduce the family of Minal codes.¹ Our codes can be defined for any dimension $\mu \geq 2$ and encode binary μ -bit messages into elements of $\mathbb{Z}_q^\mu$. However, for cryptographic implementations, we are mostly interested in low-dimensional Minal codes (e.g., $2 \leq \mu \leq 8$), for which we obtain efficient decoders. First, we discuss the motivation for higher-dimensional codes and then present the formal definition of Minal codes.

4.1 Motivation

Consider ML-KEM's mechanism for encoding a message into a polynomial. We can treat it as a two-dimensional code by pretending it encodes a pair (b_0, b_1) of message bits into coefficients $(b_0 \lfloor q/2 \rfloor, b_1 \lfloor q/2 \rfloor) \in \mathbb{Z}_q^2$. This is illustrated in Figure 1a, where dots denote the codewords, and the circles around them show the radius of minimum distance decoding.

Figure 1a highlights one limitation of ML-KEM's code: it leaves too much uncovered space under its minimum distance. To support denser codes in 2 or more dimensions, one possible solution is to use a lattice code, such as the one illustrated in Figure 1b. Lattice codes are well-known to be useful for correcting Gaussian noise, or errors that have a short Euclidean norm. However, it can be difficult to use them directly in lattice-based schemes because, in general, they are not periodic in $\mathbb{Z}_q^\mu$. To see why this is a problem, consider what happens when one adds (q, q) to the $(0, 0)$ point to the lattice shown in Figure 1b. Ideally, since ML-KEM's operations are done in $\mathbb{Z}_q$, we would like (q, q) to result in a point encoding $(0, 0)$, however, the resulting point is not a codeword, and the closest codewords to it are encodings of $(0, 1)$ and $(1, 0)$. Notice that, since the 2D view of ML-KEM's code illustrated in Figure 1a is periodic in $\mathbb{Z}_q^2$, this problem is avoided.

Previous proposals [38, 39] handle this issue by changing the parameter q to powers of 2, so it is easy to employ an 8D lattice that is periodic in $\mathbb{Z}_q^8$. While this allows such proposals to exploit the lattice structure when proving the DFR of the resulting scheme, these values of q significantly impact ML-KEM's performance, because fast NTT multiplication would no longer be available. Furthermore, the resulting scheme has larger ciphertexts.

¹The name Minal is an acronym for *Minal is not a lattice*.

To overcome such shortcomings, in what follows, we introduce a new family of higher-dimensional codes, called Minal codes, that can be seen as an intermediate between lattice codes and ML-KEM's code (see Figure 1c). By construction, we enforce that our μ -dimensional code is periodic in $\mathbb{Z}_q^\mu$. Furthermore, our code's structure is not as rigid as ML-KEM's code: our code uses a tailoring parameter that allows the position of the codewords to change. This results in codes that are more efficient at correcting errors from the particular error distribution observed for the target lattice-based schemes.

4.2 Definitions and properties

We begin with a formal definition of Minal codes.

Definition 1 (Minal code). Given an integer $\mu \geq 2$, a prime number q , and a non-negative integer $\beta < q/2$, the μ -dimensional Minal code with parameter β over alphabet $\mathbb{F}_q$ is the infinite set of points defined as $\mathcal{M} = \{\mathbf{G}\mathbf{m} + q\mathbf{z} : \mathbf{z} \in \mathbb{Z}^\mu \text{ and } \mathbf{m} \in \mathbb{Z}_2^\mu\}$, where $\mathbf{G} \in \mathbb{Z}^{\mu \times \mu}$ is the circulant matrix generated by $[q/2, \beta, 0, \dots, 0]$. Matrix $\mathbf{G}$ is called the generator matrix of $\mathcal{M}$ and β is called the tailoring parameter. We say that $\mathbf{c}$ encodes a μ -bit message $\mathbf{m} \in \mathbb{Z}_2^\mu$ when $\mathbf{c} = \mathbf{G}\mathbf{m} + q\mathbf{z}$, for some $\mathbf{z} \in \mathbb{Z}^\mu$. $\square$

A natural consequence of Definition 1 is that, by setting $(q, \beta) = (3329, 0)$, we get μ -dimensional Minal codes that are equivalent to the code used by ML-KEM. Furthermore, notice that the simple definition of Minal codes can mislead one to think that they have a linear structure. However, unlike linear codes or lattice codes, Minal codes do not even form groups, as these sets are not closed under addition. For concreteness, take two codewords $\mathbf{c}_1 = \mathbf{G}\mathbf{m}_1 + q\mathbf{z}_1$ and $\mathbf{c}_2 = \mathbf{G}\mathbf{m}_2 + q\mathbf{z}_2$. Their sum $\mathbf{c}_1 + \mathbf{c}_2 = \mathbf{G}(\mathbf{m}_1 + \mathbf{m}_2) + q(\mathbf{z}_1 + \mathbf{z}_2)$ is not always a codeword because $\mathbf{m}_1 + \mathbf{m}_2$ is not guaranteed to be in $\mathbb{Z}_2^\mu$.

Since the structure of Minal codes is periodic over $\mathbb{Z}_q^\mu$, we consider that the main representative of each codeword lies in $\mathbb{Z}_q^\mu$. In addition, to measure the distance between elements of $\mathbb{Z}_q^\mu$, we use the distance modulo q metric, defined as $\text{dist}_q(\mathbf{v}_1, \mathbf{v}_2) = \|(\mathbf{v}_1 - \mathbf{v}_2) \bmod^\pm q\|$. We can then define the minimum distance decoding under the dist_q metric as follows.

Definition 2 (Minimum distance decoding). Let $\mathcal{M}$ be a μ -dimensional Minal code over $\mathbb{F}_q$ with generator $\mathbf{G}$, and suppose we are given a vector $\mathbf{t} \in \mathbb{Z}^\mu$. A minimum distance decoder is an algorithm that finds the point $\mathbf{m} = \text{Decode}(\mathbf{t}) \in \mathbb{Z}_2^\mu$ that minimizes $\text{dist}_q(\mathbf{t}, \mathbf{G}\mathbf{m})$, that is, the distance between $\mathbf{t}$ and the codeword corresponding to $\mathbf{m}$. $\square$

Interestingly, Definition 2 implicitly defines a simple algorithm to decode a vector $\mathbf{t} \in \mathbb{Z}^\mu$: iterate over all possible $\mathbf{m} \in \mathbb{Z}_2^\mu$ to find the closest codeword to $\mathbf{t}$. While this algorithm's complexity is clearly exponential on the dimension μ , it is efficient in small dimensions. In fact, this is the decoder we use in Section 5.1 for decoding 4D Minal codes that can be applied to ML-KEM. Moreover, in Section 5.2, we also show a more efficient decoder that is specific for $\mu = 2$. Naturally, minimum distance decoding is more effective when codewords are farther apart. This motivates us to compare different codes based on the widely used minimum distance property, which is defined next for Minal codes.

Definition 3 (Minimum distance). The minimum distance of a μ -dimensional Minal code $\mathcal{M}$ over $\mathbb{F}_q$, denoted as $\text{dist}(\mathcal{M})$, is the smallest distance between different codewords in $\mathcal{M}$. Formally, we have $\text{dist}(\mathcal{M}) = \min\{\text{dist}_q(\mathbf{G}\mathbf{m}_1, \mathbf{G}\mathbf{m}_2) : \mathbf{m}_1, \mathbf{m}_2 \in \mathbb{Z}_2^\mu \text{ and } \mathbf{m}_1 \neq \mathbf{m}_2\}$. $\square$

Figure 2 shows that higher-dimensional Minal codes, when the tailoring parameter β is adequately chosen, can have much larger minimum distances. Interestingly, we observed that β has a deeper application: it can be further tuned to provide better codes for different error distributions that appear in a target scheme. For example, we observe that the best β for a 4D Minal for ML-KEM-512 is different than that for ML-KEM-1024. The next section explores this in more detail, showing tailoring procedures for finding the best value of β in a way that minimizes the code's DFR for a given error distribution.

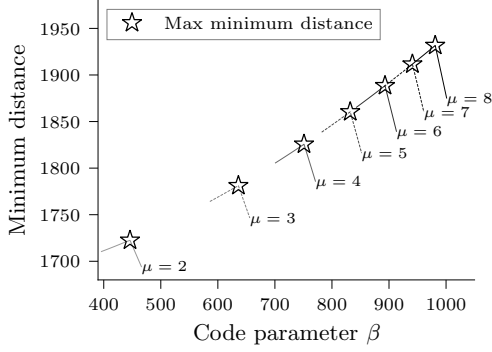


Figure 2. Effect of μ and β in the minimum distance of Minal codes for $q = 3329$.

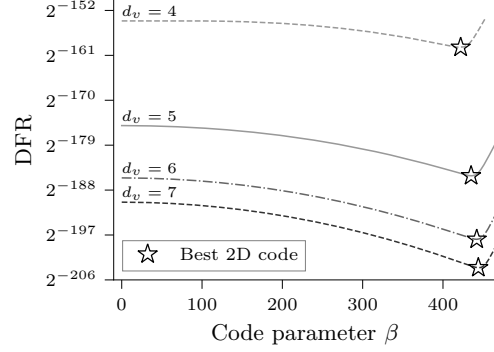


Figure 3. Effect of d_v on the DFR and on the best 2D Minal code, for ML-KEM-1024.

4.3 Tailoring Minal codes for lattice-based schemes

In most lattice-based schemes, the accumulated noise that needs to be corrected during decryption is a sum of two components. The first is an approximately normal distribution stemming from the sums of products of polynomials with a small norm. The second, present in schemes allowing ciphertext compression, is an approximately uniform component resulting from the decompression error. For concreteness, notice that the error polynomial Δm in ML-KEM indeed consists of an approximately normal factor, coming from $(\langle \mathbf{e}, \mathbf{r} \rangle - \langle \mathbf{s}, \mathbf{e}_1 + \Delta \mathbf{u} \rangle + e_2)$, and a somewhat uniform term resulting from Δv .

Although it might be tempting to pick the code with the largest minimum distance, the best choice actually depends on the overall shape² of the error distribution. For example, for a normally distributed noise, the code with the largest minimum distance is indeed the best. However, if the noise is uniform in a region, then ML-KEM’s original code would be a better choice. In what follows, we evaluate how to find the optimal tailoring parameter β under two settings. First, we consider the case in which we are able to fully determine the distribution of the noise. In this case, a simple exhaustive search for the parameter that minimizes the DFR gives us very good results, but this can only be done for small dimensions. In particular, for ML-KEM, this approach only works for 2D. We then show how to find β in larger dimensions without the need for the full noise distribution.

Tailoring in 2D using exhaustive search. In this case, we assume that the multidimensional noise distribution is efficiently computable and known. Using the noise distribution, we find the parameter β that minimizes the DFR using a simple exhaustive search. While this simple approach can be very effective, it does not scale well for dimensions higher than 2 because, in these cases, it is very expensive to compute the full noise distribution.

For a concrete example, we consider the noise distribution for ML-KEM, which can be computed using the approach described in Section 6.2. The following experiment shows how the best value of β varies depending on the noise distribution. For ML-KEM-1024, we changed parameter d_v from 1 to 12, and computed the DFR of 2D Minal codes using $\beta = 0$ to 500. Figure 3 depicts our results.

We can make two important observations. First, we clearly see that the best values of β get larger when increasing d_v , although with diminishing returns. Second, we see that the DFR improvement compared to ML-KEM’s code ($\beta = 0$) is more noticeable for higher values of d_v . Both of these stem from the fact that, by increasing d_v , we progressively lower the uniform component of the noise, increasing the effectiveness of codes with higher minimum distances. Interestingly, even for a code with only 2 dimensions, we can see that

²More formally, we can consider the level curves of the error’s probability distribution. This means that, if the error is normally or uniformly distributed, it’s overall shape is a sphere or a hypercube, respectively.

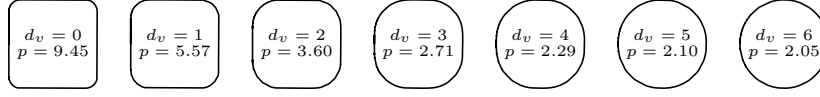


Figure 4. Level curves of 2D noise distribution corresponding to probability 2^{-128} for different values of d_v in ML-KEM-1024. The values of p show the p -norms where the shape is closely approximated by a circle.

tailoring has a major impact on the DFR, taking the DFR of ML-KEM’s original code ($\beta = 0$) from $2^{-192.4}$ down to $2^{-205.8}$ for the best tailored code ($\beta = 445$) when $d_v = 8$.

Tailoring in higher-dimensions using p -norms. We now discuss how to find a good parameter β for μ -dimensional codes without having to compute the joint distribution in μ dimensions. Our main observation is that the shape of the discrete level curves in the joint distribution of the noise can be approximated by circles in the p -norm. Concretely, if the noise distribution is approximately normal, then p is close to 2, corresponding to more circular level curves. Alternatively, if the uniform component of the noise is very strong, then p will be higher, leading to level curves shaped as squared circles.

Figure 4 shows how d_v , which is the main parameter controlling the intensity of the uniform component of the noise, impacts the overall shape of the level curves corresponding to 2^{-128} , when the other parameters are those adopted by ML-KEM-1024. It is interesting to consider both Figures 3 and 4 together, which provides a clearer picture of the relation between the optimal values of β and the overall shape of the noise distribution.

Figure 5 shows how the p -norm that best approximates the shape of the noise distribution impacts the shape of the best code. Intuitively, for a given error distribution whose shape is an approximate circle in the p -norm, the best μ -dimensional Minal code should provide a good packing of μ -dimensional spheres defined in the corresponding norm.

We then propose the following steps to find the optimal value of β .

1. Compute the 1D noise distribution $\mathcal{D}$. This is efficient for most modern schemes – e.g., Kyber [7] and Saber [15] provide scripts for this task.
2. Build the set of 2D points $P = \{(x_1, x_2) : \mathcal{D}[x_1] \cdot \mathcal{D}[x_2] \approx 2^{-\kappa}\}$. The set $P \in \mathbb{Z}^2$ approximates the level curve corresponding to probability $2^{-\kappa}$ in 2D, where $2^{-\kappa}$ is close to the DFR values we want to achieve.
3. Find the value of p that best approximates the points in P as a circle in the p -norm.
4. Find the parameter β of the μ -dimensional Minal code that maximizes the minimum distance with respect to the p -norm.

In this paper, when implementing step 2, we use $\kappa = 128$ for all parameters as it makes the procedure easier to implement and appears to provide reasonably good results. Notice that while there is an implicit independence assumption to approximate the 2D distribution using the 1D distribution $\mathcal{D}$, it does not have any security impact because we are only using it to get a reasonable value for β . The DFR of the resulting code will be derived later without requiring any independence assumptions. For step 3, we choose p that minimizes the variance of the p -norm for the points in P . Intuitively, if we treat each point in P as a radius, we want the norm in which the radii vary the least.

Step 4, which effectively finds the best tailoring parameter β , can be implemented by an exhaustive search (similar to what was done for the 2D case). However, for large dimensions μ , the computation of the minimum distance in Minal codes may be time-consuming. We can do better if we assume that the minimum distance is a unimodal function of β , i.e., starting from $\beta = 0$, the minimum distance increases until it reaches an optimal value,

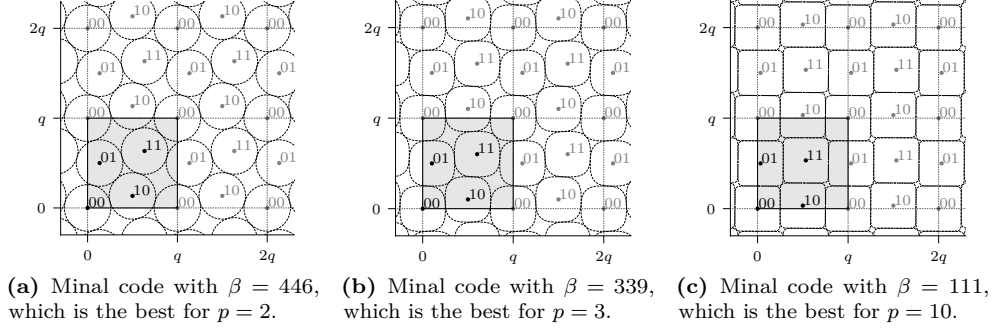


Figure 5. Best codes depending on the p -norm closest to the noise distribution.

and then it decreases. We observed this property in our empirical tests, and, in particular, it appears to hold for $\mu = 2$ dimensions and $p = 2$ (see Figure 2). However, formally proving unimodality for any dimension and p -norm does not seem trivial. Nevertheless, by assuming unimodality, we implemented step 4 using a ternary search, which is the optimal searching algorithm in this case. Since our Python implementation of this procedure provides good results for β in dimensions $\mu \leq 10$, we leave a more formal treatment of our heuristic assumptions for future work.

5 Implementation and performance

This section describes the practical aspects of our Minal codes, allowing us to achieve efficient implementations by following isochronous programming practices to protect against timing attacks. We remark that the encoding using Minal codes is rather trivial: every row of the generator matrix has only two non-null entries, so the encoding complexity per bit is independent of the dimension. Hence, this section only describes how decoding can be efficiently implemented, whereas the companion implementation contains both operations.

5.1 General decoding algorithm

For concreteness, this section describes the general decoding algorithm induced from Definition 2 by using 4D Minal codes as basis – see Algorithm 1. We emphasize, however, that the same approach can easily be extended to other dimensions while still following isochronous implementation practices. Indeed, in the accompanying code, we provide a generic implementation of the decoding algorithm that works for 2D up to 16D.

To decode a 4D point $\mathbf{t} \in \mathbb{Z}_q^4$, we need to find $\mathbf{m} \in \mathbb{Z}_2^4$ that minimizes $\text{dist}_q(\mathbf{t}, \mathbf{G}\mathbf{m}) = \|(\mathbf{t} - \mathbf{G}\mathbf{m}) \bmod^{\pm} q\|$, where $\mathbf{G}$ is the generator matrix of the Minal code. For the reference ML-KEM implementation, we can assume that $\mathbf{t} \in [-\lfloor q/2 \rfloor, \lfloor q/2 \rfloor]^4$ is already reduced modulo q and centered at zero.³ Our implementation is based on two observations. First, since the generator matrix is sparse and circulant, there are only four possible values for the coordinates of each codeword, represented by the array `CW_COORD_VALUES`. The distance between the target and each codeword can thus be computed more efficiently by using memorization of the partial squared distances between their coordinates, stored in the 4×4 matrix `dist_sqr_matrix`. The second observation is that we do not need a generic reduction algorithm, such as Barrett’s reduction, because the difference between coordinates in $[-\lfloor q/2 \rfloor, \lfloor q/2 \rfloor]$ is in $[-q, q]$ and we only need the squares of the distances. Therefore, we use a custom function `centered_mod_sqr` that, on input x , returns $(x \bmod^{\pm} q)^2$.

³In some implementations (e.g., AVX2 and pqm4), it is more convenient to use $\mathbf{t} \in [0, q)^4$, but this can be accommodated by the algorithm with minor changes.

```

1  int16_t CW_COORD_VALUES[4] = {0, CODE_BETA, KYBER_Q/2, CODE_BETA + KYBER_Q/2 - KYBER_Q};
2
3  int32_t centered_mod_sqr(int32_t value) {
4      uint32_t mask_sign = value >> 31;
5      value ^= mask_sign;
6      value += mask_sign & 1; // Result: value = abs(value)
7      value -= KYBER_Q & lower_than_mask(KYBER_Q/2, value); // Result: +-(value center_mod q)
8      return value * value; // Result: (+- (value center_mod q))^2 = (value center_mod q)^2
9  }
10 uint32_t dist_sqr_to_codeword(uint16_t idx, uint32_t dist_sqr_matrix[4][4]) {
11     int32_t a0 = dist_sqr_matrix[0][CODEWORDS[idx][0]];
12     int32_t a1 = dist_sqr_matrix[1][CODEWORDS[idx][1]];
13     int32_t a2 = dist_sqr_matrix[2][CODEWORDS[idx][2]];
14     int32_t a3 = dist_sqr_matrix[3][CODEWORDS[idx][3]];
15     return (a0 + a1 + a2 + a3) << 4 | idx; // Returns 'distance_sqr | codeword_index'
16 }
17 int decode_minal_4d(int16_t target[4]) {
18     // Build memorization matrix with square distances to target coordinates
19     uint32_t dist_sqr_matrix[4][4] = {0};
20     for (int i = 0; i < 4; i++) {
21         for (int j = 0; j < 4; j++) {
22             dist_sqr_matrix[i][j] = centered_mod_sqr(target[i] - CW_COORD_VALUES[j]);
23         }
24     }
25     uint32_t min_dist_codeword = dist_sqr_to_codeword(0, dist_sqr_matrix);
26     for (size_t i = 1; i < 16; i++) {
27         min_dist_codeword = secure_min(dist_sqr_to_codeword(i, dist_sqr_matrix), min_dist_codeword);
28     }
29     return min_dist_codeword & 0xF; // Extracts the codeword index part
30 }

```

Algorithm 1. Isochronous implementation of decoding in 4D Minal codes.

We can then implement the function `dist_sqr_to_codeword` that uses the memorization matrix to compute, for a given message index $\text{idx} \in \{0, 15\}$, the square of the distance modulo q between the target and the codeword associated with the binary expansion of idx . Using the `secure_min` function that isochronously computes the minimum between two values, the general decoding is done by iterating over each of the 16 possible messages.

The runtime of Algorithm 1 grows exponentially in the number of dimensions μ , as, in general, the loop in line 25 runs from $i = 1$ to 2^μ . However, even for more than 4 dimensions, the performance of this decoder is comparable to state-of-the-art error-correction codes used in PQC. For example, the decoder of HQC’s [31] optimized AVX2 implementation uses around 660 cycles per decoded bit. This is close to the 600 cycles per bit we observed when decoding 9D Minal codes.

5.2 Decoding 2D codewords using approximate Voronoi cells

For 2D decoding, we propose a custom algorithm that is more efficient than the generic approach from Section 5.1. We begin by observing a symmetry, illustrated in Figure 6a, that can be exploited for decoding. We can see that, by construction, the codewords of the Minal code $\mathcal{M}$ with parameter β are symmetric over the identity line, which separates the $[-q/2, q/2]^2$ square in two triangles.

Because of this property, if a point is closer to a codeword associated with $(1, 0)$ in the upper triangle, it will be closer to a codeword associated with $(0, 1)$ in the lower triangle and vice-versa – e.g., see point $\mathbf{x}$ in Figure 6a. However, we can also see that closeness to codewords associated with $(0, 0)$ and $(1, 1)$ is preserved by reflection around the identity line. Therefore, we only need a way to efficiently decode points in the lower triangle.

For the sake of building our argument for our optimized strategy, assume for a moment that q is divisible by 2. In this setting, we can construct the Voronoi cells of each codeword relevant for decoding points in the lower triangle, as illustrated in Figure 6b. By the definition of $\mathcal{M}$, the points whose Voronoi cells intersect the lower triangle, which are shown in Figure 6b, are defined as:

$$\begin{aligned}
 A &= (q/2 + \beta, q/2 + \beta), & B &= (0, 0), & C &= (\beta - q/2, \beta - q/2), \\
 D &= (q/2, \beta), & E &= (\beta, -q/2), & F &= (q/2 + \beta, -q/2 + \beta).
 \end{aligned}$$

The Voronoi cells intersecting the lower triangle can be defined by the perpendicular

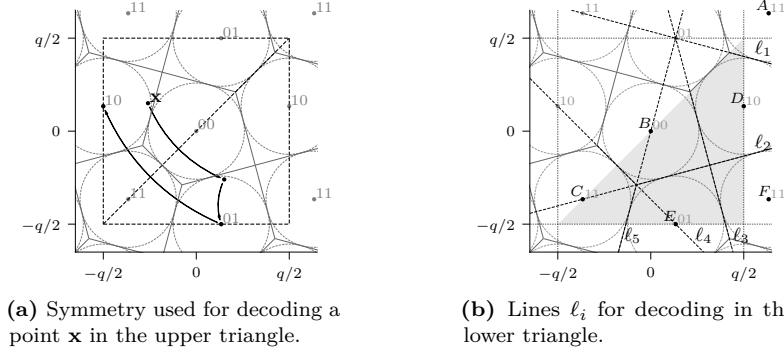


Figure 6. The geometric properties of our codes used during decoding.

bisector lines, which we call ℓ_i , between the codewords and their neighbors. First we define ℓ_1 , ℓ_4 and ℓ_5 as the bisectors between pairs (A, D) , (B, C) , and (C, E) , respectively. Now, since we assume q is even, the set of codewords $\{B, D, E, F\}$ forms a square, so line ℓ_2 is the bisector of the pairs of points (B, E) and (D, F) . Similarly, line ℓ_3 is simultaneously the bisector of both pairs (B, D) and (E, F) . This means that, for an even q , we can characterize the Voronoi cells of these codewords using only 5 lines. To effectively use these lines to decode a point (x, y) in the lower triangle, we can verify whether (x, y) is above or below ℓ_i for each i . For example, if (x, y) is above lines ℓ_4 and ℓ_2 , but below ℓ_3 , then it should be decoded as $(0, 0)$.

Now, for a real-world q , which is usually a large prime, the definition of the points is slightly different. In this case, $\{B, D, E, F\}$ is an approximate square, but not exactly one. Because the difference is rather small, the algorithm works effectively even for odd values of q . We then use this decoding approach based on approximate Voronoi cells to compute all DFR results involving 2D codes. We emphasize that there is no security issue in using this approximate decoder as long as its DFR is negligible.

Algorithm 2 shows the full algorithm for decoding using these ideas. It builds upon macros `ABOVE_Li`, that return `0xffffffff` if point (x, y) is above ℓ_i and `0x0` otherwise. Notice that the equations that define lines ℓ_i have only integer coefficients, because the coefficients of all codewords are also integers. Hence the implementation of `ABOVE_Li` based on the lines' equations uses only 32-bit integer multiplications, which most implementations, including the ones of ML-KEM, assume to be isochronous. Furthermore, a simple reflection mask is used to reflect (x, y) whenever needed, and then to reflect the result in case codewords corresponding to $(0, 1)$ or $(1, 0)$ are found.

We note that this algorithm could be made more efficient if a different square of

```

1 int decode_minal_2d(int32_t x, int32_t y) {
2     uint32_t reflect_mask = mask_lower_than(x, y); // -1 if (x < y) and 0x0 otherwise
3     int32_t x_prime = (x & ~reflect_mask) | (y & reflect_mask);
4     int32_t y_prime = (y & ~reflect_mask) | (x & reflect_mask);
5     uint8_t above11 = ABOVE_L1(x_prime, y_prime);
6     uint8_t above12 = ABOVE_L2(x_prime, y_prime);
7     uint8_t above13 = ABOVE_L3(x_prime, y_prime);
8     uint8_t above14 = ABOVE_L4(x_prime, y_prime);
9     uint8_t above15 = ABOVE_L5(x_prime, y_prime);
10    // It is unnecessary to check for (00), but: c00 = (~above13 & above12 & above14);
11    uint8_t c01 = ~above12 & ~above15 & ~above13;
12    uint8_t c10 = above12 & above13 & ~above11;
13    uint8_t c11 = above11 | (above13 & ~above12) | (above15 & ~above14);
14    c01 &= (1 ^ reflect_mask);
15    c10 &= (2 ^ reflect_mask);
16    return (c01 | c10 | c11) & 3;
17 }

```

Algorithm 2. Isochronous implementation of decoding in 2D Minal codes.

Table 3. Number of cycles for encoding and decoding under different targets.

Code	Dimension	Encoding (<code>poly_frommsg</code>)			Decoding (<code>poly_tomsg</code>)		
		AVX2	M4	A53	AVX2	M4	A53
ML-KEM’s code	1D	24	6676	441	17	3587	816
Minal code	2D	79	2017	332	418	9248	2724
Minal code	4D	62	1771	337	986	20,142	7937
Minal code	8D	44	1645	268	52,016	1,010,886	279,954

representatives was used. In particular, using the $q \times q$ square whose bottom left point is $(\lfloor -q/2 \rfloor - \epsilon, \lfloor -q/2 \rfloor - \epsilon)$, the comparison with line ℓ_1 is not necessary. However, since this would lead to a more complex description, we leave extra optimizations for future work.

5.3 Performance evaluation

To evaluate the performance of encoding and decoding operations with Minal codes, we considered three platforms for which highly optimized ML-KEM implementations are available: Intel AVX2 [7], ARM Cortex-M4 [28], and ARM Cortex-A53 [12], as described in Section 2. We integrated our isochronous implementations of 2D, 4D, and 8D Minal codes into the existing Kyber implementations in which the encoding and decoding of the full 256-bit message are done by the `poly_frommsg` and `poly_tomsg` functions, respectively. No manual optimization of the Minal code operations was done for those targets.

Table 3 shows the encoding and decoding cycle counts. Notice that, for the 2D decoding, we used our custom decoder from Section 5.2, which was about 35% faster than the general decoder in 2D (e.g., 418 cycles instead of 646 in our AVX2 setup). Unsurprisingly, decoding higher-dimensional Minal codes is more complex under all architectures. However, since the cycle count for decapsulation⁴ is much larger than this difference, the overall performance impact of Minal codes on the decapsulation procedure is very small for the 2D and 4D cases. The same cannot be said for 8D decoding, for which we leave the development of more efficient implementations as future work. The impact of our codes in ML-KEM’s decapsulation time is evaluated in Section 7.3.

5.4 On protected implementations against physical side-channel attacks

Applications that require security against side-channel attacks (SCA) typically protect their implementations using masking and shuffling countermeasures. There have been proposals for such implementations for Kyber that are directly applicable to ML-KEM [14, 25]. Although such countermeasures do not ensure resistance against SCA, their correct implementation does increase attack costs [17, 18, 35].

A potential drawback of our proposal is that the decoding step is more complex, which could encumber the application of some SCA countermeasures to it. In particular, approaches based on look-up tables (LUTs) [14] are not efficient for Minal codes. For concreteness, the decoding function $f : \{0, \dots, q-1\} \rightarrow \{0, 1\}$ for the original ML-KEM’s code can be implemented using a LUT of q rows. However, the function $f' : \{0, \dots, q-1\}^\mu \rightarrow \{0, 1\}^\mu$ for decoding μ -dimensional codes requires a LUT of q^μ rows, which can be impractical even for small values of μ . Therefore, it is important for future work to devise strategies for efficient and protected decoding of Minal codes.

6 Analyzing Minal codes’ DFR when used in ML-KEM

In this section, we discuss how to compute the DFR for our Minal codes when they are used in a lattice-based scheme. For concreteness, we focus specifically on ML-KEM, but

⁴In our AVX2 setup, decapsulation takes 20725, 31748, and 46104 cycles, for levels 1, 3, and 5.

the procedure can be extended to other schemes such as Saber [15] and NewHope [2]. While one of the main features of our analysis is that it does not require any independence assumptions on the coefficients of Δm , designers who are willing to make such assumptions can also benefit from our approach. We begin by exploring the source of the dependence between coefficients in Δm .

Consider the noise polynomial $\Delta m = \langle \mathbf{e}, \mathbf{r} \rangle - \langle \mathbf{s}, \mathbf{e}_1 + \Delta \mathbf{u} \rangle + e_2 + \Delta v$. Notice that, by definition, all coefficients from e_2 and Δv are independent. However, the coefficients of the polynomials resulting from the two dot products $\langle \mathbf{e}, \mathbf{r} \rangle$ and $\langle \mathbf{s}, \mathbf{e}_1 + \Delta \mathbf{u} \rangle$ cannot be assumed to be independent, because they are computed through sums of polynomial multiplications. If ignored, this dependence is known to cause issues in scenarios where error-correction codes are used, leading to significantly underestimating the scheme's DFR [16].

Now, let us focus on $\langle \mathbf{e}, \mathbf{r} \rangle$, which is the simplest of the two dot products that determine Δm in Kyber. It is defined as $\langle \mathbf{e}, \mathbf{r} \rangle = \mathbf{e}[0]\mathbf{r}[0] + \dots + \mathbf{e}[k-1]\mathbf{r}[k-1]$. We start by noticing that every product of polynomials $\mathbf{e}[i]\mathbf{r}[i]$ is independent of $\mathbf{e}[j]\mathbf{r}[j]$ for $j \neq i$. Also, because all $\mathbf{e}[i]$ and $\mathbf{r}[i]$ are sampled from the same $\mathcal{B}_n$, every product $\mathbf{e}[i]\mathbf{r}[i]$ follows the same distribution for all i . Therefore, in what follows, we focus our attention on the joint distribution of $\mathbf{e}[i]\mathbf{r}[i]$ for any particular i to analyze the distribution of $\langle \mathbf{e}, \mathbf{r} \rangle$.

6.1 The joint distribution of coefficients in ML-KEM's noise

We start with two results that allow us to separate the joint probability distribution of μ coefficients of a polynomial product in $R_q = \mathbb{Z}_q[x]/(x^n + 1)$ into a sum of independent distributions, provided that three conditions are met: (i) n and μ are powers of 2 with $1 < \mu < n$; (ii) the distance between adjacent pairs of coefficients is n/μ ; and (iii) the probability distribution for the coefficients of at least one of the polynomials is symmetric. In particular, by the end of this section, we characterize the μ -dimensional noise distribution

$$\Pr(\Delta m[i, i + n/2, \dots, i + (\mu - 1)n/2]),$$

so we can analyze the DFR of μ -dimensional codes without any independence assumptions.

Lemma 1. Let $n \geq 4$ be a power of 2, and let μ be a positive nontrivial divisor of n . Define $\nu = n/\mu$ and consider the probability distribution $\mathcal{P}_n$ defined over μ -tuples as

$$\mathcal{P}_n = \Pr(c[0, \nu, 2\nu, \dots, n - \nu]),$$

where the polynomial c is defined by the following experiment. Let $\mathcal{D}_1$ be any distribution over $\mathbb{Z}_q$, and $\mathcal{D}_2$ be a symmetric distribution also over $\mathbb{Z}_q$. Choose polynomials a and b in $R_q = \mathbb{Z}_q[x]/(x^n + 1)$ by taking its coefficients from distributions $\mathcal{D}_1$ and $\mathcal{D}_2$, respectively. Compute their product $c = ab \in R_q$, and output the tuple $c[0, \nu, 2\nu, \dots, (n - \nu)]$ consisting of the coefficients of c associated with powers $x^{i\nu}$, for $i = 0$ up to $\mu - 1$. Then $\mathcal{P}_n$ can be split as $\mathcal{P}_n = \mathcal{P}_{n/2} + \mathcal{P}_{n/2}$.

Proof. Let $n = 2^\ell$ for some $\ell \geq 2$. Take polynomials a and b from R_q . If $c = ab$, we can write⁵ each coefficient of the product as

$$c_i = \text{poly_to_vec}(c)[i] = \langle \text{poly_to_vec}(a), \text{negashift}_i(b) \rangle.$$

Let $\mathbf{a} = \text{poly_to_vec}(a)$ and $\mathbf{b} = \text{negashift}_0(b)$, i.e., vector $\mathbf{b}$ is the first column of the negacyclic matrix generated by the coefficients of b . Notice that, since $\mathcal{D}_2$ is symmetric, $\mathbf{b} = (b_0, -b_{n-1}, \dots, -b_1)$ has the same distribution as b . Because $n \geq 4$ is a power of 2 and μ divides n , $\mathbf{a}$ and $\mathbf{b}$ can be split into μ parts, each of length ν , such that

$$c_0 = \langle [\mathbf{a}_1, \mathbf{a}_2, \dots, \mathbf{a}_{(\mu-1)}, \mathbf{a}_\mu], [\mathbf{b}_1, \mathbf{b}_2, \dots, \mathbf{b}_{(\mu-1)}, \mathbf{b}_\mu] \rangle.$$

⁵Remember, from Section 2, that $\text{poly_to_vec}(ab)[i] = \langle \text{poly_to_vec}(a), \text{negashift}_i(b) \rangle$.

More generally, for $i = 0$ up to μ , we can write

$$c_{i\nu} = \langle [\mathbf{a}_1, \mathbf{a}_2, \dots, \mathbf{a}_{(\mu-1)}, \mathbf{a}_\mu], [-\mathbf{b}_{(\mu-i+1)}, \dots, \mathbf{b}_{(\mu-i-1)}, \mathbf{b}_{(\mu-i)}] \rangle.$$

Remember that, since μ is a nontrivial divisor of n , then $\nu = n/\mu$ is even, so we can split each $\mathbf{a}_i$ and $\mathbf{b}_i$ into two halves, such that $\mathbf{a}_i = [\mathbf{a}'_i, \mathbf{a}''_i]$ and $\mathbf{b}_i = [\mathbf{b}'_i, \mathbf{b}''_i]$, respectively. Now we write $c_{i\nu} = c'_{i\nu} + c''_{i\nu}$ where each term is defined as

$$\begin{aligned} c'_{i\nu} &= \langle [\mathbf{a}'_1, \mathbf{a}'_2, \dots, \mathbf{a}'_{(\mu-1)}, \mathbf{a}'_\mu], [-\mathbf{b}'_{(\mu-i+1)}, \dots, \mathbf{b}'_{(\mu-i-1)}, \mathbf{b}'_{(\mu-i)}] \rangle, \text{ and} \\ c''_{i\nu} &= \langle [\mathbf{a}''_1, \mathbf{a}''_2, \dots, \mathbf{a}''_{(\mu-1)}, \mathbf{a}''_\mu], [-\mathbf{b}''_{(\mu-i+1)}, \dots, \mathbf{b}''_{(\mu-i-1)}, \mathbf{b}''_{(\mu-i)}] \rangle. \end{aligned}$$

Notice that the entries of $\mathbf{a}$ and $\mathbf{b}$ appearing in terms $c'_0, c'_\nu, \dots, c'_{\mu-1}$, that come from the left halves of the blocks of length ν , are completely independent from those appearing in $c''_0, c''_\nu, \dots, c''_{\mu-1}$, which come from the right halves. Furthermore, tuples $(c'_0, c'_\nu, \dots, c'_{\mu-1})$ and $(c''_0, c''_\nu, \dots, c''_{\mu-1})$ are not only equally distributed, but, by the definition of $\mathcal{P}_j$, both of them follow the μ -dimensional distribution $\mathcal{P}_{n/2}$. Therefore, $\mathcal{P}_n = \mathcal{P}_{n/2} + \mathcal{P}_{n/2}$. $\square$

As a companion result, we now show that we can use the distribution $\mathcal{P}_n$ from Lemma 1 to characterize all distributions $\Pr(c[i, i + \nu, i + 2\nu, \dots, i + n - \nu])$, for $i = 0$ to $\nu - 1$.

Proposition 1. Let μ, ν , and n be positive integers such that $\nu = n/\mu$, and $1 < \mu < n$. Let a and b be two polynomials in $R_q = \mathbb{Z}_q[x]/(x^n + 1)$. Suppose that the coefficients of a and b are sampled from two distributions $\mathcal{D}_1$ and $\mathcal{D}_2$, respectively, and assume that $\mathcal{D}_2$ is symmetric. If $c = ab$ is their product, then, for all $0 \leq i < \nu$, we have

$$\Pr(c[i, i + \nu, i + 2\nu, \dots, i + n - \nu]) = \Pr(c[0, \nu, 2\nu, \dots, n - \nu]).$$

Proof. Since $c = ab$, each coefficient of c is given as $c_i = \langle \text{poly_to_vec}(a), \text{negashift}_i(b) \rangle$. Notice that this means that the μ -tuple $c[i, i + \nu, i + 2\nu, \dots, i + n - \nu]$ is completely defined by the sequence of vectors $S_i = (\text{negashift}_i(b), \dots, \text{negashift}_{(i+n-\nu)}(b))$. But remember that the elements of b come from a symmetric distribution. Therefore, by the definition of the negacyclic shifts, the sequence S has exactly the same distribution as $S_0 = (\text{negashift}_0(b), \dots, \text{negashift}_{(n-\nu)}(b))$. $\square$

We notice that Lemma 1 is analogous to the *polynomial splitting for recovery* used in NewHope's original analysis [3, §C]. The main difference is that we provide a broader presentation for supporting a more direct construction of the joint distribution, whose effective computation is not needed in related works.

Let us now discuss the applicability of Lemma 1 to ML-KEM. Fix parameters $q = 3329$ and $n = 256$. Consider the two dot products $\langle \mathbf{e}, \mathbf{r} \rangle$ and $\langle \mathbf{s}, \mathbf{e}_1 + \Delta \mathbf{u} \rangle$ that appear in the computation of Δm . The first one is done with polynomials whose coefficients are taken from the centered binomial $\mathcal{B}_{\eta_1}$, which is symmetric. In the second one, elements from $\mathbf{s}$ are also drawn from $\mathcal{B}_{\eta_1}$. Therefore, we can swap the operands of the commutative product so that all of the lemma's hypotheses are satisfied. We now provide a simple corollary that explicitly states the distribution of the coefficients of the ML-KEM noise.

Corollary 1. Fix integers μ and ν such that $\nu = n/\mu$ and $1 < \mu < n$. Let $\mathcal{P}_{\text{prod}}^{(\phi_a, \phi_b)}$ denote the probability distribution of a product $c = ab$ of two polynomials $a, b \in \mathbb{Z}_q[x]/(x^\mu + 1)$, with coefficients selected according to distributions ϕ_a and ϕ_b , respectively. Let $\mathcal{D}_{\Delta \mathbf{u}}$ denote the distribution of the coefficients of $\Delta \mathbf{u}$ and $\mathcal{P}_{(\Delta v + e_2)}$ be the probability distribution of $(\Delta v + e_2)[i, i + \nu, \dots, i + n - \nu]$. Then, by Lemma 1 and Proposition 1, we have:

$$\Delta m[i, i + \nu, \dots, i + n - \nu] \sim \frac{kn}{\mu} \mathcal{P}_{\text{prod}}^{(\mathcal{B}_{\eta_1}, \mathcal{B}_{\eta_1})} + \frac{kn}{\mu} \mathcal{P}_{\text{prod}}^{(\mathcal{B}_{\eta_2}, \mathcal{B}_{\eta_1} + \mathcal{D}_{\Delta \mathbf{u}})} + \mathcal{P}_{(\Delta v + e_2)}. \quad \square$$

```

1: procedure DFR 4D(code  $\mathcal{M}$ )
2:   Compute the 4D Voronoi cells of the main codewords of  $\mathcal{M}$ 
3:    $p_{\text{failure}} \leftarrow 0$  ▷ Accumulates the decoding error probability for  $\mathcal{M}$ 
4:   for each main codeword  $\mathbf{c} \in \mathcal{M} \cap \mathbb{Z}_q^4$  do
5:      $\text{vor}(\mathbf{c}) \leftarrow$  Voronoi cell centered in  $\mathbf{c}$ 
6:      $\text{neigh}(\mathbf{c}) \leftarrow$  set of codewords whose cells share a hyperplane with  $\text{vor}(\mathbf{c})$ 
7:     for each adjacent codeword  $\mathbf{v}$  in  $\text{neigh}(\mathbf{c}) \in \mathcal{M}$  do
8:        $\hat{\mathbf{v}} \leftarrow \mathbf{v} - \mathbf{c}$ 
9:       Compute the 1D distribution  $\Pr(\langle \Delta m[0, n/4, 2n/4, 3n/4], \hat{\mathbf{v}} \rangle)$ 
10:       $p_{\text{failure}} \leftarrow p_{\text{failure}} + \frac{1}{16} \Pr\left(\langle \Delta m[0, n/4, 2n/4, 3n/4], \hat{\mathbf{v}} \rangle \geq \frac{\|\hat{\mathbf{v}}\|^2}{2}\right)$ 
11:   return  $\frac{n}{4} p_{\text{failure}}$  ▷ Union bound over the  $n/4 = 64$  tuples

```

Algorithm 3. Computation of the DFR for a 4D Minal code $\mathcal{M}$.

When μ is sufficiently small, we can calculate the base distributions over coefficient pairs, $\mathcal{P}_{\text{prod}}^{(\mathcal{B}_{\eta_1}, \mathcal{B}_{\eta_1})}$ and $\mathcal{P}_{\text{prod}}^{(\mathcal{B}_{\eta_2}, \mathcal{B}_{\eta_1} + \mathcal{D}_{\Delta u})}$, by enumerating the corresponding polynomials in $\mathbb{Z}_q/(x^\mu + 1)$ and computing their products while keeping track of the associated probabilities. Also, the product rule can be used to directly compute $\mathcal{P}_{(\Delta v + e_2)}$, as the coefficients in both Δv and e_2 are all independent.

The above analysis has an important consequence on how we encode messages using μ -dimensional Minal codes. Since $n = 256$ in ML-KEM, we are bound to use codes whose dimension is a power of 2. Additionally, since we need to compute the base μ -dimensional distributions, the complexity grows exponentially, making it harder to use $\mu \geq 8$ for ML-KEM parameters. Moreover, to use Corollary 1 when evaluating the DFR of μ -dimensional codes applied to ML-KEM, we must encode each μ -bit string in the message $\mathbf{m} \in \mathbb{Z}_2^n$ into coefficients separated by n/μ entries. In the following sections, we show how to evaluate the DFR when using 2D and 4D Minal codes in ML-KEM.

6.2 Computing the DFR using 2D Minal codes

The core observation allowing us to compute the DFR for 2D Minal codes is that it is possible to use Lemma 1 to fully compute the 2D joint noise distribution $\Delta m[i, i + n/2]$, which is then used for computing the DFR. While the Python scripts [19] provided by the Kyber team are fast enough to compute the distribution of a single coefficient of Δm , their approach is highly inefficient when computing sums of joint distributions. To address this problem, we implemented a custom 2-dimensional FFT with multiprecision complex arithmetic using the MPC [21] and MPFR [22] libraries. Running in a standard PC using 6 threads, the computation of the joint probability distribution $\Pr(\Delta m[i, i + n/2])$ with 260 bits of precision takes less than 3 minutes for each parameter set.

6.3 Computing the DFR using 4D Minal codes

If we try to use the approach from Section 6.2 to compute the DFR in the 4D setting, we need to compute the joint distribution $\Pr(\Delta m[0, n/4, n/2, 3n/4])$. The problem is that the 4D FFTs, which have to be computed with padding so that the convolutions do not cause cyclic interference, would incur prohibitively large memory and processing costs. Our solution is then to make a different use of Lemma 1 in a way that is akin to how the DFR of NewHope’s first variant was computed [3]. Algorithm 3 briefly describes the 4D DFR computation, while a detailed explanation is given in what follows.

First, we define the parameters q and β of the Minal code $\mathcal{M}$ whose DFR we want to evaluate. Usually, the field size q is defined by the cryptographic scheme (e.g., $q = 3329$ for ML-KEM) and β can be found using the p -norm approach discussed in Section 4.3. Using these parameters, we can compute the 4D Voronoi cells for all main codewords using the Quickhull [10] algorithm from the Qhull library [11]. From the 4D Voronoi cells, we can see

which are the Voronoi-relevant vectors of each codeword, i.e., the neighboring codewords that characterize the Voronoi cells of a given codeword.

Based on the Voronoi cells, we define a function $\text{neigh}(\mathbf{c})$ that, for a main codeword $\mathbf{c} \in \mathcal{M} \cap \mathbb{Z}_q^4$, returns the set of points whose Voronoi cells share a common hyperplane with the Voronoi cell centered in $\mathbf{c}$. Let $p_{\text{error}}(\mathbf{c}, \mathbf{v})$ denote the probability that $\mathbf{c}' = \mathbf{c} + \Delta m[0, n/4, 2n/4, 3n/4]$ is closer to $\mathbf{v}$ than to $\mathbf{c}$. The DFR is then upper-bounded by

$$\text{DFR} \leq \frac{1}{16} \sum_{\mathbf{c} \in \mathcal{M} \cap \mathbb{Z}_q^4} \left(\sum_{\mathbf{v} \in \text{neigh}(\mathbf{c})} p_{\text{error}}(\mathbf{c}, \mathbf{v}) \right),$$

where the $1/16$ factor accounts for the probability of getting each codeword $\mathbf{c}$.

We know, from elementary linear algebra, that $\|\mathbf{c}' - \mathbf{c}\| \geq \|\mathbf{c}' - \mathbf{v}\|$ if, and only if, $\langle \mathbf{c}', \mathbf{v} - \mathbf{c} \rangle \geq \frac{1}{2}(\|\mathbf{v}\|^2 - \|\mathbf{c}\|^2)$. Then, expanding $\mathbf{c}'$, we can compute $p_{\text{error}}(\mathbf{c}, \mathbf{v})$ as

$$p_{\text{error}}(\mathbf{c}, \mathbf{v}) = \Pr \left(\langle \Delta m[0, n/4, 2n/4, 3n/4], \mathbf{v} - \mathbf{c} \rangle \geq \frac{1}{2}\|\mathbf{v} - \mathbf{c}\|^2 \right).$$

While we cannot compute the noise distribution $\Pr(\Delta m[0, n/4, 2n/4, 3n/4])$ in 4D, we can compute $p_{\text{error}}(\mathbf{c}, \mathbf{v})$ as follows. Consider the following vectors and their associated probability distributions: $\mathbf{x}_1 \sim \mathcal{P}_{\text{prod}}^{(\mathcal{B}_{\eta_1}, \mathcal{B}_{\eta_1})}$, $\mathbf{x}_2 \sim \mathcal{P}_{\text{prod}}^{(\mathcal{B}_{\eta_2}, \mathcal{B}_{\eta_1})}$, $\mathbf{x}_3 \sim \mathcal{D}_{\Delta \mathbf{u}}$, and $\mathbf{x}_4 \sim \mathcal{P}_{(\Delta v + e_2)}$. Define probability distributions $\mathcal{D}_1, \mathcal{D}_2, \mathcal{D}_3, \mathcal{D}_4$ such that each $\mathcal{D}_i = \Pr \langle \mathbf{x}_i, \mathbf{v} - \mathbf{c} \rangle$, for $i = 1$ up to 4. Then, by Corollary 1 and the linearity of the dot product, we have

$$\Pr(\langle \Delta m[0, n/4, 2n/4, 3n/4], \mathbf{v} - \mathbf{c} \rangle) = \frac{kn}{\mu} \mathcal{D}_1 + \frac{kn}{\mu} \mathcal{D}_2 + \mathcal{D}_3 + \mathcal{D}_4.$$

Notice that each $\mathcal{P}_i$ is efficiently computable because the dimension 4 is relatively small and is a simple 1D distribution. Now, to compute the multiple convolutions above, we can again use arbitrary precision FFTs. We notice that, because $\mathbf{v} - \mathbf{c}$ may have large coordinates, the padding required for the FFT-based convolution is relatively high. Therefore, we use Bailey's [9] trick that transforms an FFT over a large vector into a 2D FFT with shorter vectors to make the implementation more efficient. For a given ML-KEM parameter set, together with the 4D Minal code parameters, our implementation gives the corresponding DFR in about 1 hour in a standard computer using 6 threads.

7 Applications to compact ML-KEM instantiations

This section shows a key benefit of our proposed codes: with them, ML-KEM parameter sets associated with shorter ciphertext sizes can achieve lower decryption failure rates (DFR), thus making them viable. We begin by remarking that the size of a ciphertext $(\mathbf{c}_u, c_v)$ in ML-KEM is completely determined by the parameters n, k, d_u , and d_v as $\lceil \frac{1}{8}n(kd_u + d_v) \rceil = 32(kd_u + d_v)$ bytes. Hence, to obtain shorter ciphertexts without changing the core parameters n and k , we need to use smaller parameters for d_u and d_v than those adopted by the current standard. In this section, we show how this can be accomplished for each security level supported by ML-KEM.

7.1 The DFR targets for each security level

Since our goal is to propose parameters that reduce ciphertext or public-key sizes without compromising the DFR, we now explicitly state which DFR values we consider viable for each security level. We start by noting that NIST did not define such DFR targets, but only requires them to be negligibly small values. This gave KEM designers a lot of

flexibility to propose parameters offering good balance between sizes, security, and DFR. However, it also resulted in very different DFRs among different schemes: code-based schemes such as BIKE [4] and HQC [31] target very low DFRs of $2^{-\lambda}$, where λ is the scheme’s security in bits, while Kyber and Saber are more permissive. Namely, in round 1, Kyber mentions the DFR of 2^{-140} as a target for all levels [5]. Then, in Kyber’s round 2 specification, the authors state that the DFR target was at most 2^{-160} for all security levels [6, §1.5]. This was later relaxed in round 3, with DFR targets of 2^{-128} for level 1, and kept at 2^{-160} for levels 3 and 5 [7, §1.4 and §4.4], as the authors perceived the previous DFR levels as too conservative. Conversely, Saber [15] does not state DFR targets, but only achieved DFRs of 2^{-120} , 2^{-136} and 2^{-165} for NIST security levels 1, 3, and 5.

We could define the DFR targets to be exactly those obtained in the latest version of Kyber (or ML-KEM), and have ciphertext compression for Level 5, or public-key compression for Level 1, as shown in the next section. However, this seems too restrictive since Kyber enjoyed some flexibility when changing parameters between rounds, and NIST considers Saber to be secure even with its higher DFR levels. Hence, we sought to provide useful insights on possible parameters while imposing a limit on how much our DFR can deviate from the state of the art in lattice-based schemes. As a result, we hereby define our DFR targets for each security level as the maximum between the DFR targets mentioned in the latest Kyber specification and the concrete DFR provided by Saber parameter sets. This gives us $\text{DFR}_1 = 2^{-120}$, $\text{DFR}_3 = 2^{-136}$, and $\text{DFR}_5 = 2^{-160}$, for levels 1, 3, and 5, respectively. These targets are then used in the next section to select viable parameters.

7.2 Finding compression parameters that yield shorter ciphertexts

First, we consider ML-KEM settings with values of (d_u, d_v) , allowing for shorter ciphertexts than the current standard. We then compute the DFR for the regular ML-KEM 1D code, and for our 2D and 4D Minal codes tailored for the error distribution induced by these parameters. Then, we can select points with shorter ciphertext sizes, as long as their DFR lies below the target for each security level.

Figure 7 shows our results. Note that results for the experiment described above are the regular (d_u, d_v) points, i.e., points marked as $(d_u, d_v)^*$ or $(d_u, d_v)^\dagger$ are not part of this experiment, as they require additional changes that are described later in this section. We can see that the impact of our codes is greater in Level 5, as there are multiple points using 2D and 4D codes with significantly shorter ciphertexts – namely, up to 8% compression. This results from the fact that Level 5 parameters, in general, use higher values of d_v , so the uniform part of the noise is less relevant, and our codes are better at error correction. To allow for an easier comparison of the actual numbers, we collect in Table 4 the relevant parameters discussed in this section.

Now, to obtain compressed ciphertexts for levels 1 and 3, we proceed as follows. First, remember that the security evaluation of ML-KEM-512 uses not only the LWE hardness but also considers the deterministic compression noise when evaluating the Core-SVP hardness associated with the ciphertext security, which is akin to also considering the LWR in the security analysis. This allows ML-KEM-512 to use a slightly lower value⁶ for η_2 , without compromising security, because the extra noise introduced by the compression accounts for the decrease in η_2 . If we extend this idea further, we can use even more aggressive compression factors than the values $(d_u, d_v) = (10, 4)$ used in ML-KEM-512 and ML-KEM-768, and get rid of η_2 completely. This idea is actually not new, as it is even mentioned in Kyber’s round 1 specification [5, §6.4.6]. However, for this approach to be successful for ML-KEM, one needs to use a better error-correction code because it is more difficult to deal with the higher compression noise, which is uniform in nature, than with the noise from the centered binomials.

⁶Remember that η_2 defines the binomial distribution for vector $\mathbf{e}_1$ and polynomial e_2 used in encryption.

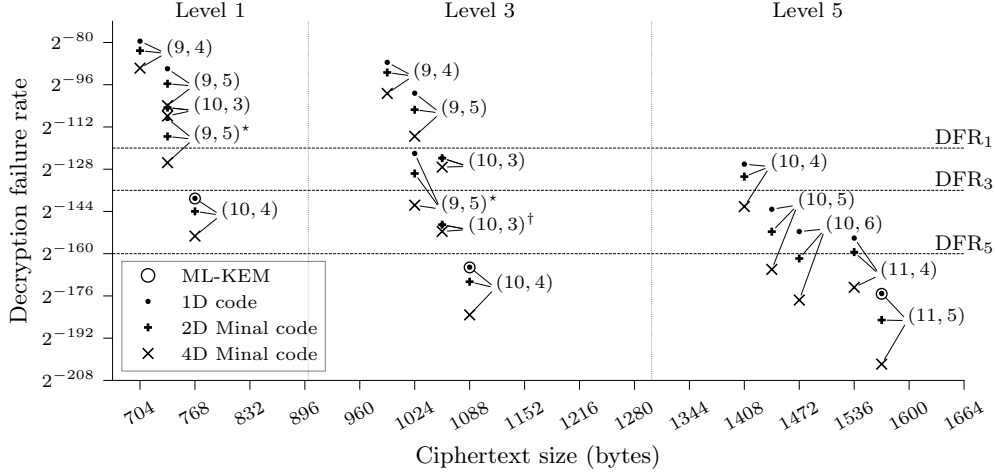


Figure 7. The impact of our 2D and 4D Minal codes in obtaining viable compression parameters (d_u, d_v) allowing for shorter ciphertexts. Parameters marked with a star (*) or a dagger (†) require changing η_2 to 0 and 1, respectively.

Table 4. Relevant parameters allowing for ciphertext compression. Non-viable codes whose DFR lie above the DFR target are marked with a slash.

NIST Level	DFR target	d_u	d_v	η_2	Is a standard parameter set?	Requires LWR?	Ciphertext size (bytes)	Minimum 1D	2D	4D
1	2^{-120}	10	4	2	ML-KEM-512	Yes	768	$2^{-139.1}$	$2^{-144.0}$	$2^{-153.3}$
		9	5	0	No	Yes	736	$2^{-108.9}$	$2^{-115.6}$	$2^{-125.5}$
3	2^{-136}	10	4	2	ML-KEM-768	No	1088	$2^{-165.2}$	$2^{-170.6}$	$2^{-183.2}$
		10	3	1	No	Yes	1056	$2^{-149.2}$	$2^{-149.2}$	$2^{-151.6}$
		9	5	0	No	Yes	1024	$2^{-122.1}$	$2^{-129.6}$	$2^{-141.6}$
5	2^{-160}	11	5	2	ML-KEM-1024	No	1568	$2^{-175.2}$	$2^{-185.1}$	$2^{-201.8}$
		11	4	2	No	No	1536	$2^{-154.2}$	$2^{-159.4}$	$2^{-172.7}$
		10	6	2	No	No	1472	$2^{-151.7}$	$2^{-161.8}$	$2^{-177.6}$
		10	5	2	No	No	1440	$2^{-143.3}$	$2^{-151.7}$	$2^{-165.9}$

The result of this approach is illustrated in Figure 7 by the points $(9, 5)^*$ and $(10, 3)^\dagger$ that define $\eta_2 = 0$ and $\eta_2 = 1$, respectively. Note that these are only considered for levels 1 and 3. We can see that our 4D Minal codes allow for viable points, providing 4% and 6% shorter ciphertexts for both levels 1 and 3, respectively. We remark that all parameters provide the same Core-SVP hardness as ML-KEM’s standards for the corresponding security level, as computed by the Kyber team’s security scripts [19].

7.3 Proposed parameters and their performance impact

Section 5.3 shows that the decoding operation of Minal codes is significantly more costly than that of ML-KEM’s 1D code, due to the higher-dimensional nature of our proposal. However, we emphasize that the time taken by our isochronous decoding algorithms, up to 4D, are still orders of magnitude lower than the full decapsulation time. Moreover, since our Minal codes allow $\eta_2 = 0$ to be a viable setting, together with (d_u, d_v) in levels 1 and 3, the full setup also benefits from not having to make the additional samplings related to η_2 in the encryption procedure.

To show this reduced impact, we use portable isochronous implementations of the Minal code operations, rather than optimizing it for the given targets, while integrating our code into highly optimized implementations for AVX2 [7], Cortex M4 [28] and Cortex-A53 [12].

Table 5. ML-KEM instantiations with compressed ciphertexts and speedups for full encapsulation and decapsulation.

NIST Level	(d_u, d_v, η_2)	Minal code	DFR	Ciphertext compression	Encaps speedup			Decaps speedup		
					AVX2	M4	A53	AVX2	M4	A53
1	(9, 5, 0)	4D ($\beta = 745$)	$2^{-125.5}$	4.17%	0.96	1.14	1.13	0.99	1.08	1.02
3	(9, 5, 0)	4D ($\beta = 741$)	$2^{-141.6}$	5.88%	1.07	1.10	1.07	1.05	1.07	1.01
5	(10, 6, 2)	2D ($\beta = 442$)	$2^{-161.8}$	6.12%	0.99	1.00	1.00	1.00	1.00	0.99
5	(10, 5, 2)	4D ($\beta = 741$)	$2^{-165.9}$	8.16%	1.01	1.00	1.00	0.99	0.99	0.97

Table 6. ML-KEM instances with compressed public keys and speedups for full encapsulation and decapsulation. Notice that the values of η_2 are the same as those in the standard.

NIST Level	(d_u, d_v, η_2)	Minal code	DFR	Public key compression	Encaps speedup			Decaps speedup		
					AVX2	M4	A53	AVX2	M4	A53
1	(10, 4, 3)	4D ($\beta = 722$)	$2^{-139.8}$	8.00%	1.00	1.00	0.97	0.94	0.97	0.93
3	(10, 4, 2)	4D ($\beta = 722$)	$2^{-162.9}$	8.11%	0.98	0.99	0.96	0.97	0.98	0.95
5	(11, 5, 2)	4D ($\beta = 745$)	$2^{-174.0}$	8.16%	1.01	0.99	0.98	0.99	0.99	0.97

The results are presented in Table 5, which shows the speedup⁷ of our proposal compared to the encapsulation and decapsulation times for ML-KEM implementations using standard parameters. As expected, the performance impact is minor. For levels 1 and 3, there is even a significant speedup resulting from the fact that some of the sampling operations are not needed in encryption⁸ since $\eta_2 = 0$. For level 5, no speedup was observed since there is no change to η_2 , but the performance impact is still negligible, especially when 2D codes are used. Note that all parameters in Table 5 provide the same core-SVP hardness as the standard ML-KEM parameters, for the corresponding security levels, as computed by Kyber’s security scripts [19].

If we compare our results with the ones obtained by Saliba et al. [38, 39] for 8D lattice codes (see Table 2), we can see the power of tailoring. For concreteness, consider level 5: even the low-dimensional 2D Minal codes already provide ciphertext compression, requiring a simple change in the compression factors; in contrast, the 8D codes from [38, 39] require changes to core ML-KEM parameters, and actually increase the ciphertext size.

7.4 An overview of additional applications of our codes

Due to space limitations, the results presented so far focus more on applications to ciphertext compression, which are arguably more immediately applicable to ML-KEM. However, we argue that there are other important applications to which our codes also have a relevant impact, as briefly discussed next.

Eliminating the LWR assumption from ML-KEM-512. Recall that ML-KEM-512 relies on a hardness assumption similar to LWR to achieve its claimed security using $\eta_2 = 2 < \eta_1$. Using our 4D codes with $\beta = 722$, though, we can instantiate ML-KEM-512 using $\eta_2 = \eta_1 = 3$, and get a DFR of $2^{-135.6}$ that is very close to the one observed for ML-KEM-512 (i.e., $2^{-139.1}$). This would not only eliminate the need for the LWR assumption, but it would also result in an easier specification by removing the need for one additional parameter. Moreover, it would allow for simpler vectorized implementations of the sampling, since only one function would be needed for generating all centered binomial values.

⁷For each security level, the speedup is defined as the ratio between the execution time of the original ML-KEM and that of our proposal.

⁸The decapsulation also calls the encryption procedure due to the reencryption step of the FO transform.

Public-key compression with minor performance impact. Kyber’s round 1 specification [5] allowed compression of the public key, which is useful in ephemeral-key settings. Using a slightly different formulation [13, Thm. 1] of the noise polynomial Δm to take public key compression into account, we extended our analysis for this case.

Table 6 shows our results when public-key compression is enabled. We consider the compression of coefficients in $\mathbb{Z}_{3329}$ from 12 bits, as used in the standard (no compression), to 11 bits. Notice that, in this setting, the sender does an extra decompression operation for the public key (which is a very efficient task), while this step is not needed for the receiver. The performance impact of our proposal is then estimated by adding the decompression cost to the encapsulation time. Notice that, in this setting, there is no change to $\eta_2 > 0$. Hence, in Levels 1 and 3, the decapsulation is slower than what is shown in Table 5, but still minor, especially for Cortex-M4. Our 4D Minal codes allow for 8% compression of ML-KEM public keys while keeping the DFR very close to the current standards without compression (namely, the impact is less than 2 bits), for all NIST security levels.

Exploring the parameter space considering our codes. Higher-dimensional Minal codes empower designers of lattice-based KEMs by providing a richer set of trade-offs between failure rate and both ciphertext and public key sizes. Remarkably, we showed (see Table 5) that Minal codes not only enable more compact schemes, but can even lead to significantly faster ML-KEM variants, as they allow parameter sets with $\eta_2 = 0$ to be secure and have DFR values lower than the corresponding targets. Therefore, our results suggest that Minal codes can be explored much beyond if we consider ML-KEM variants with different core parameters, possibly resulting in significantly more compact and faster schemes.

8 Conclusion and future work

We present a new family of error-correction codes called Minal codes. By proposing a novel way to model the accumulated noise in lattice-based schemes as μ -dimensional circles under different p -norms, we show how Minal codes can be tailored to improve their error correction capability for each particular application without requiring any change to the target scheme’s parameters. We then demonstrate how to compute the decryption failure rate (DFR) when applied to ML-KEM, and the corresponding gains, through an analysis that could be adapted to most modern lattice-based schemes.

This work also raises several questions. First, we believe the most important theoretical question is whether we can build more efficient decoders, possibly borrowing results from the decoding of lattice codes and adapting them to our case. However, there are other interesting research directions, such as formalizing and possibly proving some of our heuristic⁹ assumptions used for tailoring, or trying to find even better error-correction codes by considering more complex constructions with additional parameters.

With respect to the practical side, it would be interesting to devise efficient implementations using SIMD instructions for CPUs supporting AVX2/AVX512 or ARM NEON. We believe there is great potential for more efficient high-dimensional decoding when approximate decoders are considered, if these can be shown to still provide negligible DFR. Extending our DFR analysis to ML-KEM using 8D Minal codes would also be interesting. If, as in our present work, one is not willing to make simplifying independence assumptions because these can yield underestimated DFR values and security concerns [16], the main challenge is the increased complexity of computing the joint 8D base distributions (before the self-convolutions). Finally, it would be relevant to understand how Minal codes behave when decoding is implemented with countermeasures against physical side-channel attacks, and also analyze our code’s impact on prominent lattice-based schemes such as Saber [15] and NewHope [3].

⁹The heuristic assumptions are only used for tailoring and have no security impact.

References

- [1] Gorjan Alagic, Jacob Alperin-Sheriff, Daniel Apon, David Cooper, Quynh Dang, Yi-Kai Liu, Carl Miller, Dustin Moody, Rene Peralta, Ray Perlner, Angela Robinson, and Daniel Smith-Tone. *Status report on the first round of the NIST post-quantum cryptography standardization process*. US Department of Commerce, National Institute of Standards and Technology, 2019. doi:10.6028/NIST.IR.8240. Cited on page 2.
- [2] Erdem Alkim, Roberto Avanzi, Joppe W. Bos, Leo Ducas, Antonio de la Piedra, Thomas Poppelmann, Peter Schwabe, and Douglas Stebila. NewHope (Version 1.1): Algorithm Specifications And Supporting Documentation. <https://newhopecrypto.org/resources.shtml>, Apr 2020. Cited on pages 2 and 14.
- [3] Erdem Alkim, Léo Ducas, Thomas Pöppelmann, and Peter Schwabe. Post-quantum key exchange – A new hope. In *25th USENIX Security Symposium (USENIX Security 16)*, pages 327–343, 2016. URL: https://www.usenix.org/system/files/conference/usenixsecurity16/sec16_paper_alkim.pdf. Cited on pages 2, 5, 15, 16, and 21.
- [4] Nicolas Aragon, Paulo S. L. M. Barreto, Slim Bettaleb, Loïc Bidoux, Olivier Blazy, Jean-Christophe Deneuville, Philippe Gaborit, Santosh Ghosh, Shay Gueron, Tim Güneysu, Carlos Aguilar-Melchor, Rafael Misoczki, Edoardo Persichetti, Jan Richter-Brockmann, Nicolas Sendrier, Jean-Pierre Tillich, Valentin Vasseur, and Gilles Zémor. BIKE: Bit flipping key encapsulation, 2022. URL: https://bikesuite.org/files/v5.0/BIKE_Spec.2022.10.04.1.pdf. Cited on page 18.
- [5] Roberto Avanzi, Joppe Bos, Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, John Schanck, Peter Schwabe, Gregor Seiler, and Damien Stehlé. CRYSTALS-Kyber: Algorithm specifications and supporting documentation, 2017. <https://pq-crystals.org/kyber/data/kyber-specification.pdf>. Cited on pages 18 and 21.
- [6] Roberto Avanzi, Joppe Bos, Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, John Schanck, Peter Schwabe, Gregor Seiler, and Damien Stehlé. CRYSTALS-Kyber: Algorithm specifications and supporting documentation (version 2.0), 2019. <https://pq-crystals.org/kyber/data/kyber-specification-round2.pdf>. Cited on page 18.
- [7] Roberto Avanzi, Joppe W. Bos, Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, John Schanck, Peter Schwabe, Gregor Seiler, and Damien Stehlé. CRYSTALS-Kyber: Algorithm specifications and supporting documentation (version 3.02), 2021. <https://pq-crystals.org/kyber/data/kyber-specification-round3-20210804.pdf>. Cited on pages 1, 3, 9, 13, 18, and 19.
- [8] Hayo Baan, Sauvik Bhattacharya, Scott Fluhrer, Oscar Garcia-Morchon, Thijs Laarhoven, Ronald Rietman, Markku-Juhani O. Saarinen, Ludo Tolhuizen, and Zhenfei Zhang. Round5: Compact and fast post-quantum public-key encryption. In *Post-Quantum Cryptography: 10th International Conference, PQCrypto 2019, Chongqing, China, May 8–10, 2019 Revised Selected Papers 10*, pages 83–102. Springer, 2019. doi:10.1007/978-3-030-25510-7_5. Cited on page 2.
- [9] David H. Bailey. FFTs in external or hierarchical memory. *The journal of Supercomputing*, 4:23–35, 1990. doi:10.1007/BF00162341. Cited on page 17.
- [10] C. Bradford Barber, David P. Dobkin, and Hannu Huhdanpaa. The Quickhull algorithm for convex hulls. *ACM Transactions on Mathematical Software (TOMS)*, 22(4):469–483, 1996. doi:10.1145/235815.235821. Cited on page 16.

- [11] C. Bradford Barber, David P. Dobkin, and Hannu Huhdanpaa. The Quickhull algorithm for convex hulls. *ACM Trans. on Mathematical Software*, 22(4):469–483, Dec 1996. <http://www.qhull.org>. Cited on page 16.
- [12] Hanno Becker, Vincent Hwang, Matthias J. Kannwischer, Bo-Yin Yang, and Shang-Yi Yang. Neon NTT: Faster Dilithium, Kyber, and Saber on Cortex-A72 and Apple M1. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2022(1):221–244, Nov. 2021. Artifact available at <https://artifacts.iacr.org/tches/2022/a2>. doi:10.46586/tches.v2022.i1.221-244. Cited on pages 13 and 19.
- [13] Joppe Bos, Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, John M Schanck, Peter Schwabe, Gregor Seiler, and Damien Stehlé. CRYSTALS-Kyber: A CCA-secure module-lattice-based KEM. In *2018 IEEE European Symposium on Security and Privacy (EuroS&P)*, pages 353–367. IEEE, 2018. doi:10.1109/EuroSP.2018.00032. Cited on page 21.
- [14] Joppe W. Bos, Marc Gourjon, Joost Renes, Tobias Schneider, and Christine van Vredendaal. Masking Kyber: First- and higher-order implementations. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2021(4):173–214, Aug. 2021. URL: <https://tches.iacr.org/index.php/TCHES/article/view/9064>, doi:10.46586/tches.v2021.i4.173-214. Cited on page 13.
- [15] Jan-Pieter D’Anvers, Angshuman Karmakar, Sujoy Sinha Roy, and Frederik Vercauteren. SABER: Mod-LWR based KEM (Round 3 Submission). *Submission to the NIST post-quantum project*, 2020. URL: <https://www.esat.kuleuven.be/cosic/pqcrypto/saber/files/saberspecround3.pdf>. Cited on pages 2, 9, 14, 18, and 21.
- [16] Jan-Pieter D’Anvers, Frederik Vercauteren, and Ingrid Verbauwhede. The impact of error dependencies on Ring/Mod-LWE/LWR based schemes. In Jintai Ding and Rainer Steinwandt, editors, *Post-Quantum Cryptography*, pages 103–115, Cham, 2019. Springer International Publishing. doi:10.1007/978-3-030-25510-7_6. Cited on pages 1, 2, 5, 14, and 21.
- [17] Jeroen Delvaux. Roulette: A diverse family of feasible fault attacks on masked Kyber. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2022(4):637–660, Aug. 2022. URL: <https://tches.iacr.org/index.php/TCHES/article/view/9835>, doi:10.46586/tches.v2022.i4.637-660. Cited on page 13.
- [18] Elena Dubrova, Kalle Ngo, Joel Gärtner, and Ruize Wang. Breaking a fifth-order masked implementation of CRYSTALS-Kyber by copy-paste. In *Proceedings of the 10th ACM Asia Public-Key Cryptography Workshop, APKC ’23*, page 10–20, New York, NY, USA, 2023. Association for Computing Machinery. doi:10.1145/3591866.3593072. Cited on page 13.
- [19] L. Ducas and J. Schanck. Security estimation scripts for Kyber and Dilithium. Available: <https://github.com/pq-crystals/security-estimates>, 2021. Cited on pages 4, 5, 16, 19, and 20.
- [20] Jan-Pieter D’Anvers, Qian Guo, Thomas Johansson, Alexander Nilsson, Frederik Vercauteren, and Ingrid Verbauwhede. Decryption failure attacks on IND-CCA secure lattice-based schemes. In *Public-Key Cryptography–PKC 2019: 22nd IACR International Conference on Practice and Theory of Public-Key Cryptography, Beijing, China, April 14–17, 2019, Proceedings, Part II 22*, pages 565–598. Springer, 2019. doi:10.1007/978-3-030-17259-6_19. Cited on pages 1 and 5.

- [21] Andreas Enge, Mickaël Gastineau, Philippe Théveny, and Paul Zimmermann. MPC – A library for multiprecision complex arithmetic with exact rounding, December 2022. <http://www.multiprecision.org/mpc/>. Cited on page 16.
- [22] Laurent Fousse, Guillaume Hanrot, Vincent Lefèvre, Patrick Pélicier, and Paul Zimmermann. MPFR: A multiple-precision binary floating-point library with correct rounding. *ACM Transactions on Mathematical Software (TOMS)*, 33(2):13–es, 2007. doi:10.1145/1236463.1236468. Cited on page 16.
- [23] Eiichiro Fujisaki and Tatsuaki Okamoto. Secure integration of asymmetric and symmetric encryption schemes. In *Annual International Cryptology Conference*, pages 537–554. Springer, 1999. doi:10.1007/3-540-48405-1_34. Cited on page 4.
- [24] Qian Guo, Thomas Johansson, and Jing Yang. A novel CCA attack using decryption errors against LAC. In Steven D. Galbraith and Shiho Moriai, editors, *Advances in Cryptology – ASIACRYPT 2019*, pages 82–111, Cham, 2019. Springer International Publishing. doi:10.1007/978-3-030-34578-5_4. Cited on pages 1 and 5.
- [25] Daniel Heinz, Matthias J. Kannwischer, Georg Land, Thomas Pöppelmann, Peter Schwabe, and Amber Sprenkels. First-order masked Kyber on ARM Cortex-M4. Cryptology ePrint Archive, Paper 2022/058, 2022. URL: <https://eprint.iacr.org/2022/058>. Cited on page 13.
- [26] Dennis Hofheinz, Kathrin Hövelmanns, and Eike Kiltz. A modular analysis of the Fujisaki-Okamoto transformation. In Yael Kalai and Leonid Reyzin, editors, *Theory of Cryptography*, pages 341–371, Cham, 2017. Springer International Publishing. doi:10.1007/978-3-319-70500-2_12. Cited on page 4.
- [27] Matthias Kannwischer, Peter Schwabe, Douglas Stebila, and Thom Wiggers. Improving software quality in cryptography standardization projects. In *IEEE European Symposium on Security and Privacy, EuroS&P - Workshops*, pages 19–30, Los Alamitos, CA, USA, Jun 2022. IEEE Computer Society. URL: <https://eprint.iacr.org/2022/337>, doi:10.1109/EuroSPW55150.2022.00010. Cited on page 3.
- [28] Matthias J Kannwischer, Joost Rijneveld, Peter Schwabe, and Ko Stoffelen. pqm4: Testing and benchmarking NIST PQC on ARM Cortex-M4. In *Second PQC Standardization Conference: University of California, Santa Barbara and co-located with Crypto 2019*, pages 1–22, 2019. URL: <https://github.com/mupq/pqm4/tree/c32bcd017b202d418c9135e2df77be73a69044a0>. Cited on pages 3, 13, and 19.
- [29] Shuiyin Liu and Amin Sakzad. Lattice codes for CRYSTALS-Kyber, September 2023. URL: <http://arxiv.org/abs/2308.13981>. Cited on pages 2, 5, and 6.
- [30] Xianhui Lu, Yamin Liu, Zhenfei Zhang, Dingding Jia, Haiyang Xue, Jingnan He, Bao Li, and Kunpeng Wang. LAC: Practical Ring-LWE based public-key encryption with byte-level modulus. Cryptology ePrint Archive, Paper 2018/1009, 2018. URL: <https://eprint.iacr.org/2018/1009>. Cited on page 2.
- [31] Carlos Aguilar Melchor, Nicolas Aragon, Slim Bettaieb, Loïc Bidoux, Olivier Blazy, Jurjen Bos, Jean-Christophe Deneuville, Arnaud Dion, Philippe Gaborit, Jérôme Lacan, Edoardo Persichetti, Jean-Marc Robert, Pascal Véron, and Gilles Zémor. Hamming Quasi-Cyclic: HQC, 2021. https://pqc-hqc.org/doc/hqc-specification_2021-06-06.pdf. Cited on pages 11 and 18.

- [32] National Institute of Standards and Technology. FIPS203: Module-lattice-based key-encapsulation mechanism standard. Federal Inf. Process. Stds. (NIST FIPS), National Institute of Standards and Technology, Aug 2024. <https://doi.org/10.6028/NIST.FIPS.203>. Cited on pages 1, 2, and 4.
- [33] Thomas Plantard, Arnaud Sipasseuth, Willy Susilo, and Vincent Zucca. Tight bound on NewHope failure probability. *IEEE Transactions on Emerging Topics in Computing*, 10(4):1955–1965, 2022. [doi:10.1109/TETC.2021.3138951](https://doi.org/10.1109/TETC.2021.3138951). Cited on page 5.
- [34] Raspberry Pi. Raspberry Pi Zero 2 W. Available: <https://www.raspberrypi.com/products/raspberry-pi-zero-2-w/>, 2024. Cited on page 3.
- [35] Prasanna Ravi, Thales Paiva, Dirmanto Jap, Jan-Pieter D’Anvers, and Shivam Bhasin. Defeating low-cost countermeasures against side-channel attacks in lattice-based encryption: A case study on CRYSTALS-Kyber. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2024(2):795–818, Mar. 2024. URL: <https://tches.iacr.org/index.php/TCHES/article/view/11447>, [doi:10.46586/tches.v2024.i2.795-818](https://doi.org/10.46586/tches.v2024.i2.795-818). Cited on page 13.
- [36] Oded Regev. On lattices, learning with errors, random linear codes, and cryptography. *Journal of the ACM (JACM)*, 56(6):1–40, 2009. [doi:10.1145/1568318.1568324](https://doi.org/10.1145/1568318.1568324). Cited on page 1.
- [37] Markku-Juhani O. Saarinen. HILA5: On reliability, reconciliation, and error correction for Ring-LWE encryption. In *Selected Areas in Cryptography–SAC 2017: 24th International Conference, Ottawa, ON, Canada, August 16-18, 2017, Revised Selected Papers 24*, pages 192–212. Springer, 2018. [doi:10.1007/978-3-319-72565-9_10](https://doi.org/10.1007/978-3-319-72565-9_10). Cited on page 2.
- [38] Charbel Saliba. *Error correction and reconciliation techniques for lattice-based key generation protocols*. PhD thesis, CY Cergy Paris Université, 2022. URL: <https://theses.hal.science/tel-03718212v1/document>. Cited on pages 2, 5, 6, and 20.
- [39] Charbel Saliba, Laura Luzzi, and Cong Ling. A reconciliation approach to key generation based on Module-LWE. In *2021 IEEE International Symposium on Information Theory (ISIT)*, pages 1636–1641, 2021. [doi:10.1109/ISIT45174.2021.9517882](https://doi.org/10.1109/ISIT45174.2021.9517882). Cited on pages 2, 5, 6, and 20.
- [40] STMicroelectronics. STM32 Nucleo-144 development board with STM32F439ZI MCU, supports Arduino, ST Zio and morpho connectivity. Available: <https://www.st.com/en/evaluation-tools/nucleo-f439zi.html>, 2024. Cited on page 3.